

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

**Design of a Multi-Agent AI Pipeline
for Evidence Collection from
Illegal Streaming Sites**

By: Ahmed ARFAOUI

Academic supervisor: [Academic Supervisor]

Corporate Internship Supervisor: Wael Hkiri

LOGO ENTREPRISE

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname** : Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname** : Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname** : [Academic Supervisor]

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of an agentic evidence-collection system for illegal streaming websites. The work was carried out during an internship at Soft Stars for the client beIN SPORTS. The objective was to support rights-enforcement workflows by visiting target websites, classifying their pages, extracting hosting and stream evidence, and preserving auditable outputs such as screenshots, traces, and structured results.

The internship produced two related artifacts: an exploratory workflow used to validate browser-agent behavior on real targets, and a **Python**-based reference architecture designed to improve maintainability, observability, memory, evaluation, and scalability. The report presents these artifacts progressively, starting from the general evidence-collection problem and moving toward the concrete implementation and evaluation details.

The proposed architecture organizes evidence collection around specialized reasoning agents, browser automation tools, structured outputs, trace storage, screenshot capture, and evaluation metrics. The database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs; it is not the enterprise production database of the client organization.

Acknowledgements

I would like to thank my academic and professional supervisors for their guidance throughout this internship. I also thank the Soft Stars team for the technical discussions, feedback, and trust that made this work possible.

Contents

1	Introduction	1
1.1	Host organization – Soft Stars	1
1.2	Sports piracy and the evidence collection challenge	1
1.3	Objectives and contributions	1
1.4	Report structure	2
2	Methodology, Requirements Analysis, and Design Constraints	4
2.1	Methodology options considered	4
2.1.1	Agile	4
2.1.2	CRISP-DM	4
2.1.3	Design Science Research Methodology (DSRM)	5
2.2	Chosen methodology: DSRM with iterative prototyping	6
2.3	DSRM applied to this project	6
2.4	Project evolution timeline	7
2.5	Functional and non-functional requirements	8
3	Technical Background	10
3.1	Streaming Website Structures	10
3.1.1	The normal page flow	10
3.1.2	Landing pages	11
3.1.3	Hosting pages	11
3.1.4	Embedded pages	12
3.2	DOM Structures and Web Page Representation	12
3.2.1	A simplified DOM tree	12
3.2.2	DOM signals on streaming pages	13
3.3	Streaming Protocols and Embedded Players	14
3.3.1	Common embedding patterns	14
3.3.2	HLS and MPEG-DASH	14
3.3.3	Tokens, cookies, referers, and provider checks	15
3.4	Browser Automation, Scraping, and Runtime Obstacles	15
3.4.1	Puppeteer as a browser control layer	15
3.4.2	Ads, cookies, popups, and other annoyances	16
3.4.3	Why screenshots and network logs matter	16
3.5	Agentic Evidence Collection Concepts	16
3.5.1	Agents and ReAct-style loops	17

3.5.2	Prompt engineering, tokens, and token usage	17
3.5.3	Workflow automation tools	18
3.5.4	LangChain, tools, and MCP servers	18
3.6	Chapter Summary	19
4	System Architecture	20
4.1	System Purpose and Architecture Reading Path	20
4.2	Global Runtime Architecture	20
4.3	End-to-End Agent Pipeline	23
4.4	Agent Communication and Orchestrator Handoffs	24
4.5	Agent-by-Agent Design	26
4.5.1	Orchestrator Agent	26
4.5.2	Classification Agent	26
4.5.3	Landing-Page Agent	27
4.5.4	Hosting-Page Agent	29
4.5.5	Embedded-Page Agent	29
4.5.6	Provider Analysis and Email Generation	30
4.6	Data, Trace, and Reviewability View	30
4.7	Chapter Summary	32
5	Implementation	33
5.1	Part 1: n8n Workflow Prototype	33
5.1.1	Purpose of the n8n phase	33
5.1.2	n8n workflow screenshots	33
5.1.3	Limitations of the n8n implementation	35
5.2	Part 2: Implemented Reference Solution	35
5.2.1	FastAPI Backend	36
5.2.2	Next.js Operator Console	37
5.2.3	MCP Browser Containers and Puppeteer Tools	37
5.2.4	Agents, Models, Memory, and Prompts	37
5.2.4.1	Design constraints implemented in the agent runtime	39
5.2.4.2	Prompt evolution in the implementation	39
5.2.4.3	Agent design principles	40
5.2.4.4	Model behavior, token usage, and rate limits	41
5.2.4.5	Prompt engineering and stopping behavior	41
5.2.5	PostgreSQL Database and Observability	41
5.3	Part 3: Deployment	43
5.4	Implementation Summary	43

6	Evaluation and Testing	45
6.1	Agent Evaluation and Website Regression	45
6.1.1	Test Protocol	45
6.1.2	Batch Acceptance Rule	46
6.1.3	Agent Use Cases	47
6.1.4	Prompt Regression Results	47
6.1.5	Dashboard Snapshot	48
6.2	Model Evaluation, KPI Selection, and Cache	50
6.2.1	KPI Formulas	50
6.2.2	Model Selection Criteria	50
6.2.3	LLM Model Comparison	50
6.2.4	Cost Estimation	53
6.2.5	Provider Intelligence Test	54
6.3	Tool Evaluation	54
6.3.1	Tool Success Criteria	54
6.3.2	Tool Regression Findings	56
6.4	Chapter Conclusion	56
7	Conclusion and Perspectives	58
7.1	General Conclusion	58
7.2	Main Contributions	58
7.3	Technical Lessons	59
7.3.1	Prompt structure matters	59
7.3.2	Browser evidence must be layered	59
7.3.3	Fallbacks are part of the architecture	59
7.3.4	Evaluation must be regression-oriented	59
7.4	Limitations	59
7.5	Future Work	60
7.5.1	Screenshot annotation layer	60
7.6	Final Remarks	60

List of Figures

1.1	Report reading map following a general-to-specific structure.	3
2.1	Methodology selection logic: Agile and CRISP-DM used as supporting lenses, DSRM used as the main backbone.	4
2.2	CRISP-DM structure and Agile iterations used as supporting methodology lenses.	5
2.3	DSRM adopted for the project, with Agile-style short iterations inside each phase.	6
2.4	Project evolution timeline from domain orientation to evaluated reference archi- tecture.	8
3.1	Common navigation flow from event listing to hosting page and embedded player.	10
3.2	Visual comparison of the three page types used by the extraction workflow: landing discovers candidates, hosting activates and observes the network, and embedded is a fallback player context.	11
3.3	Simplified DOM tree showing how a browser represents an HTML document. .	13
3.4	Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.	14
3.5	Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH. . . .	15
3.6	General ReAct-style loop used by browser agents.	17
3.7	General relationship between an LLM agent, profile-scoped MCP tools, and structured browser observations.	18
4.1	Use-case overview of the evidence-collection platform.	21
4.2	Component architecture based on the documented UI, backend, agent runtime, MCP tooling, persistence, and external-service boundaries.	22
4.3	End-to-end agent architecture at workflow scale.	23
4.4	Sequence diagram for launching, executing, persisting, and reviewing a work- flow run.	23
4.5	Agent interconnection graph based on the architecture documentation.	24
4.6	Orchestrator routing flowchart and handoff decisions.	25
4.7	Prompt compilation and memory inputs for each browser-facing specialist. . . .	26
4.8	Orchestrator-agent design card.	26
4.9	Sequence diagram for orchestrator handoff, trace recording, and route updates.	27
4.10	Classification-agent design card.	27
4.11	Classification-agent sequence: inspect page state and return a routing decision.	28
4.12	Landing-page-agent design card.	28
4.13	Landing-agent sequence: convert listing-page evidence into downstream queues.	28

4.14	Hosting-page-agent design card.	29
4.15	Hosting-agent sequence: activate watch pages and return stream or iframe evidence.	29
4.16	Embedded-page-agent design card.	30
4.17	Embedded-agent sequence: stay in the player context and harvest final media evidence.	30
4.18	Provider-analysis and email-generation design card.	31
4.19	Runtime inputs and outputs for browser-facing agents based on the agent documentation.	31
5.1	Role of the n8n workflow as a feasibility playground.	33
5.2	n8n workflow screenshots: full sequential workflow and the four specialist agent workflows.	34
5.3	How n8n limitations shaped the reference implementation.	36
5.4	Five implementation areas of the reference solution.	36
5.5	Current Next.js operator-console screens used for launch, review, telemetry, provider enrichment, and runtime configuration.	38
5.6	Browser tool execution loop through MCP and Puppeteer.	39
5.7	Prompt evolution from one-pass intent prompts to evidence-driven agent loops.	40
5.8	Agent reasoning loop with memory and tool feedback.	41
5.9	Simplified database relationship map for run review.	42
5.10	Azure Container Apps Job deployment using one container for worker, Puppeteer MCP, and Chrome.	44
6.1	Regression-batch improvement across the main browser-agent KPIs.	49

List of Tables

2.1	Methodology selection matrix for the evidence-collection project.	5
2.2	DSRM phase mapping to project activities and outputs.	7
2.3	Functional requirements derived from the evidence-collection workflow.	8
2.4	Non-functional requirements and implementation responses.	9
3.1	Streaming-site page taxonomy and evidence role.	12
3.2	DOM signals used to understand streaming-site pages.	13
3.3	Browser-level obstacles and handling strategies.	16
4.1	Use-case responsibilities across operator and administration workflows.	21
4.2	Global runtime boundaries used by the reference architecture.	22
4.3	Agent and tool responsibility map.	25
4.4	Runtime data boundaries and persisted review outputs.	31
5.1	n8n prototype agent roles and implementation lessons.	34
5.2	n8n prototype limitations and engineering consequences.	35
5.3	Backend API responsibilities in the reference implementation.	36
5.4	Current Next.js operator-console surfaces and backend dependencies.	37
5.5	Browser automation engine comparison for the MCP tool layer.	38
5.6	Puppeteer MCP tool groups and responsibilities.	39
5.7	Specialist agent objectives and assigned tool surfaces.	39
5.8	Runtime constraints and their implementation responses.	40
5.9	Model-runtime concerns handled by the agent platform.	42
5.10	Prompt engineering rules enforced by the specialist agents.	42
5.11	PostgreSQL observability records and their implementation role.	43
5.12	Landing-agent deployment scope and responsibilities.	44
6.1	Evaluation test matrix executed over the website regression batch.	46
6.2	Website-level acceptance criteria for regression success.	46
6.3	Agent use cases and website-level success signals.	47
6.4	Prompt regression history and batch-level behavioral focus.	48
6.5	Use-case KPI results measured on the website regression batch.	48
6.6	Dashboard telemetry values used as operational evidence.	49
6.7	KPI formulas for model, agent, and website-batch evaluation.	50
6.8	Model selection criteria for the regression batch.	51
6.9	LLM model comparison.	51

6.10	Observed cost, token, and cache metrics from the dashboard.	53
6.11	Provider-intelligence telemetry used to validate stream URL enrichment. . . .	54
6.12	Tool-family evaluation criteria for batch-level testing.	55
6.13	Observed tool-health telemetry from the dashboard.	55
6.14	Tool-level regressions found through website-batch testing.	56
6.15	Tool-description improvements measured against the website batch.	56
7.1	Final summary of project contributions and outcomes.	58

1 Introduction

1.1 Host organization – Soft Stars

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship was carried out within Soft Stars in the context of sports media protection. The project was connected to the broader problem of monitoring illegal streaming websites and improving the way evidence is collected from them. At this stage of the report, the system is introduced only from a general business and engineering perspective. The detailed process, implementation choices, and evaluation are developed progressively in the following chapters.

1.2 Sports piracy and the evidence collection challenge

Premium sports broadcasting depends on exclusive distribution rights. Broadcasters pay for the right to transmit competitions in specific territories, on specific platforms, and under controlled commercial conditions. Illegal streaming websites undermine this model by redistributing protected events without authorization, often through domains that change frequently and through pages designed to be difficult to monitor manually.

The first practical challenge in this context is not only finding suspicious websites. It is collecting evidence that is precise, traceable, and useful. A rights holder needs to know which page was reachable, when it was observed, what content or player it exposed, and which technical indicators can support a later review or enforcement action. Screenshots, visible page state, embedded players, network indicators, and stream references can all become important, but they are often hidden behind dynamic interfaces, popups, nested frames, language variation, and short-lived links.

This makes evidence collection an engineering problem. Manual review is slow and inconsistent when the number of targets grows. Simple scraping is too fragile for modern streaming pages, because the useful evidence may appear only after the page loads scripts, reacts to user actions, or redirects through several layers. The general goal is therefore to support a more reliable way of observing these websites, collecting evidence, and keeping enough context for the result to be auditable.

1.3 Objectives and contributions

The objective of this project is to study and design a technical approach for evidence collection from illegal sports streaming websites. The work moved from a manual and case-by-case

process toward a more structured process that can inspect target pages, preserve useful observations, and present results in a form that can be reviewed.

Two artifacts were produced for different purposes. First, an n8n workflow was used to validate the operational idea quickly on real dynamic websites. Second, a Python reference architecture was designed to make the same workflow more maintainable and observable through specialist agents, versioned prompts, profile-scoped browser tools, memory, caching, traces, screenshots, and model usage records. The Python architecture should therefore be read as the engineering reference and evaluation platform for the agentic approach, not as a claim that the full company workflow was replaced during the internship.

The main objectives are:

- understand the operational context of sports piracy monitoring and the type of evidence required by rights-protection teams;
- analyze the technical behavior of illegal streaming websites, including dynamic pages, embedded players, redirects, and multilingual layouts;
- define a maintainable evidence-collection process that can adapt to landing pages, hosting pages, embedded players, redirects, popups, and repeated layout patterns;
- design a system architecture that separates observation, extraction, persistence, memory, caching, review, and evaluation concerns;
- evaluate the approach on representative targets and document its limits.

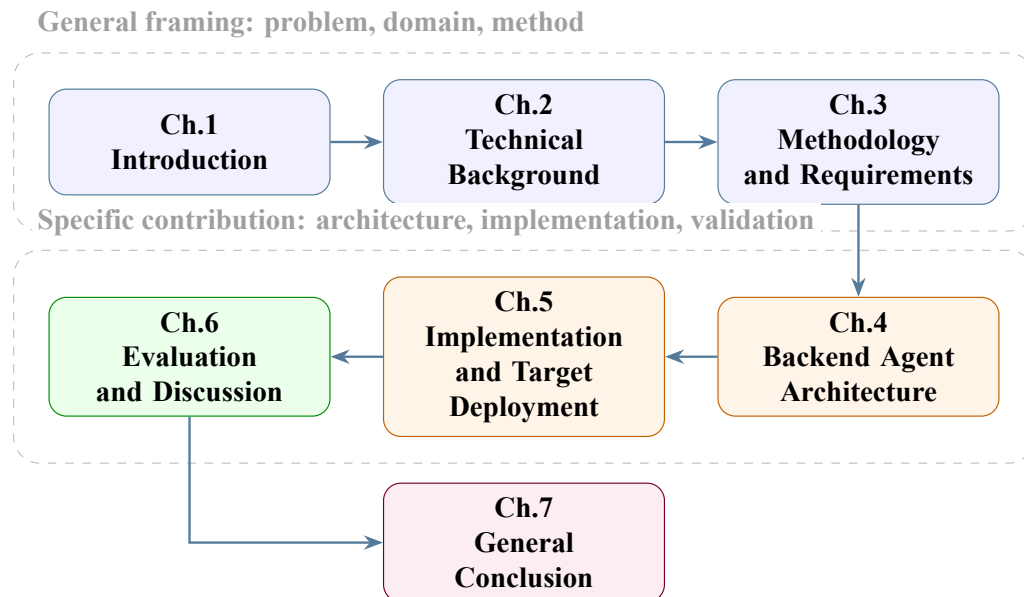
The main contributions of the internship are:

- a structured analysis of the evidence-collection problem in the sports piracy domain;
- a validated n8n prototype showing that LLM-guided browser observation can follow dynamic streaming-site flows;
- a technical architecture for collecting, storing, and reviewing evidence from suspicious streaming websites through specialized agents;
- an explicit description of how prompts, browser tools, short-term memory, long-term memory, caching, and context-window handling support the agent workflow;
- an evaluation of the approach based on observed behavior, collected traces, and failure cases;
- a report structure that moves from the general business problem toward the specialized implementation and validation work.

1.4 Report structure

This report is organized from general context to specialized implementation. Chapter 2 presents the methodology, requirements analysis, and design constraints used to frame the project. Chapter 3 introduces the technical background needed to understand streaming websites, browser-based observation, automation tools, agentic extraction, and media protocols. Chapter 4 describes the proposed architecture and explains how the main system responsibilities are organized. Chapter 5 details the implementation path, from the n8n exploration workflow

to the concrete software components and the final operational scope. Chapter 6 evaluates the system through agent, model, cache, cost, and tool testing. Chapter 7 concludes the report.



The report moves from the piracy evidence-collection problem toward the designed backend agent artifact and its evaluation.

Figure 1.1: Report reading map following a general-to-specific structure.

2 Methodology, Requirements Analysis, and Design Constraints

This chapter presents the methodological frame adopted for the internship, explains why Design Science Research Methodology (DSRM) was selected, maps the six DSRM phases to the concrete work carried out, and derives the requirements and constraints that shaped the artifact described in Chapters 4 and 5. Because the internship delivered and evaluated technical artifacts rather than a predictive model, the methodological emphasis is on artifact design, demonstration, and evaluation.

2.1 Methodology options considered

The internship combined software engineering, browser automation, prompt design, and evidence-oriented investigation. Three methodological options were considered: Agile, CRISP-DM, and Design Science Research Methodology (DSRM).

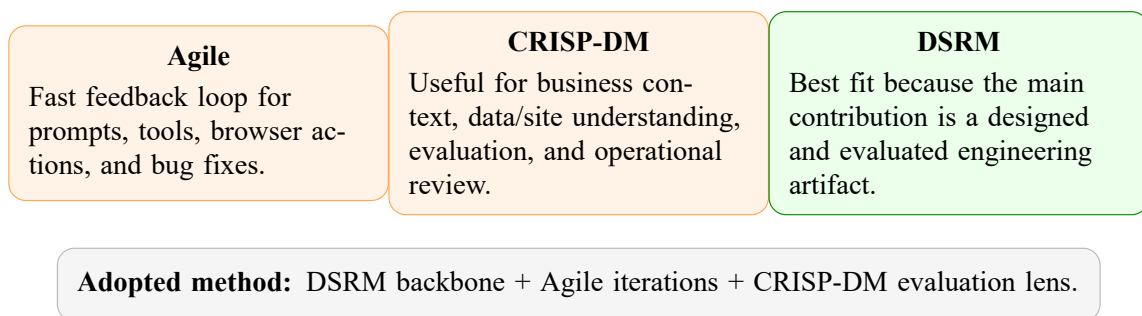


Figure 2.1: Methodology selection logic: Agile and CRISP-DM used as supporting lenses, DSRM used as the main backbone.

2.1.1 Agile

Agile practices matched the day-to-day execution rhythm of the internship. Short cycles of implementation, run inspection, failure analysis, and prompt or tool revision were essential because illegal streaming sites change quickly and often expose new obstacles only during live testing. Agile was therefore used as an execution discipline: it helped turn each failed run into a small correction to a prompt, browser tool, selector strategy, or evaluation check. It was not used as the full research methodology, because it does not by itself explain how the artifact is justified, demonstrated, and evaluated.

2.1.2 CRISP-DM

CRISP-DM [19] offers a clear sequence from business understanding to data understanding, preparation, modeling, evaluation, and operational review. Parts of that logic were useful

for organizing observations: understanding the enforcement context, studying page structures, preparing test cases, evaluating evidence quality, and discussing operational deployment. However, the project was not a classical data-mining lifecycle and did not aim to optimize a predictive model on a labeled dataset. CRISP-DM is therefore kept as an evaluation and operational-review lens.

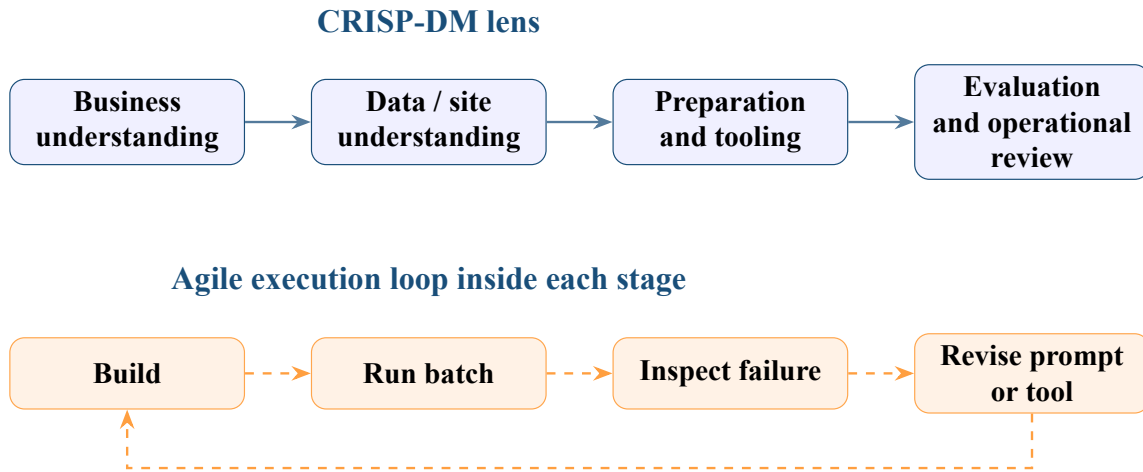


Figure 2.2: CRISP-DM structure and Agile iterations used as supporting methodology lenses.

2.1.3 Design Science Research Methodology (DSRM)

Design science research focuses on the creation and evaluation of artifacts that solve relevant organizational problems [20, 21]. This framing matches the internship more closely because the central deliverable is an engineering artifact: an agentic browser-based evidence-collection pipeline composed of prompts, tools, browser traces, screenshots, structured outputs, and evaluation metrics.

Methodology	Strength for this project	Limitation for this project
Agile	Excellent short-cycle execution and adaptation during prompt, tool, and browser refinement	Less explicit about how to justify and evaluate a technical artifact as a research contribution
CRISP-DM	Useful discipline for scoping, evaluation, and operationalization discussion	Dataset/model-centered framing is not the main logic of this internship
DSRM	Artifact-centered framing that explicitly links problem, design, demonstration, and evaluation	Needs complementary iteration rhythm for fast engineering feedback

Table 2.1: Methodology selection matrix for the evidence-collection project.

2.2 Chosen methodology: DSRM with iterative prototyping

DSRM was adopted as the overall methodology, while Agile-style short iterations were used inside each phase to refine prompts, tools, and browser behavior based on observed failures. This means that the report follows the DSRM logic, but the practical work did not move through the phases only once. Each failed run could send the work back to the objectives, design, or evaluation step.

DSRM fits the project better than CRISP-DM for three reasons. First, the project is artifact-centered, not dataset-centered or model-centered: the main work was to design a reliable evidence-collection system under adversarial web conditions. Second, the main output is a working and proposed system architecture together with a validated agent workflow, not a trained predictive model. Third, evaluation focuses on tool behavior, evidence quality, trace completeness, extraction success and failure patterns, cost, and robustness rather than on statistical accuracy against a fixed labeled benchmark.

CRISP-DM remains a useful secondary lens for discussing operational context and evaluation discipline, but it is not the primary methodological backbone of the report. Figure 2.3 summarizes the adopted DSRM view, while Figure 2.2 already placed the supporting Agile and CRISP-DM logic under the methodology options.

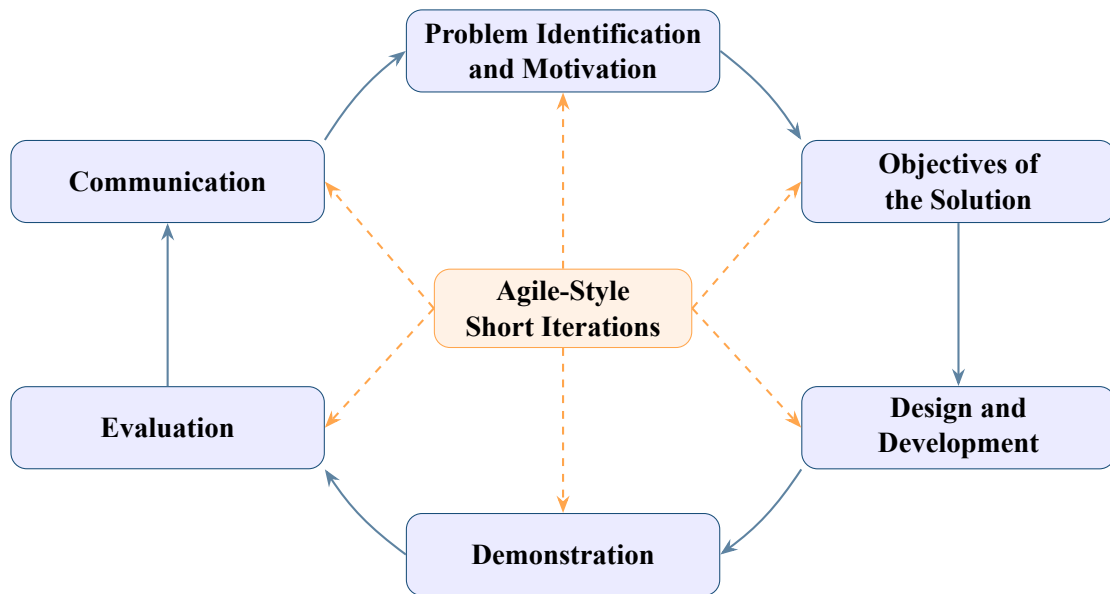


Figure 2.3: DSRM adopted for the project, with Agile-style short iterations inside each phase.

2.3 DSRM applied to this project

Following Peffers et al. [21], the project was organized around six DSRM phases. Table 2.2 maps each phase to the concrete internship work and its main outputs.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Illegal streaming evidence collection is dynamic, adversarial, multilingual, and difficult for static scrapers because evidence may appear only after runtime interaction, iframe traversal, or player activation.	Problem framing and motivation for the artifact
Objectives of the solution	Define that the artifact must classify pages, extract host/embed/stream evidence, capture screenshots, preserve traces, and calculate evaluation metrics that support reviewable evidence collection.	Solution objectives and acceptance criteria
Design and development	Build an exploratory workflow and a validated single-agent path; derive a Python/FastAPI reference architecture for modularity, observability, memory, evaluation, and future scalability.	Validated workflow plus reference architecture
Demonstration	Run the artifact on representative streaming-site families: landing pages, hosting pages, embedded player pages, and iframe/popup/tokenized-stream cases.	Demonstration cases and observed behaviors
Evaluation	Measure prompt behavior, tool usage, extraction success and failure, token and cost metrics, cache behavior, trace quality, and practical limitations.	Evaluation traces, metrics, and findings
Communication	Produce the internship report, diagrams, architecture documentation, and implementation roadmap.	Communicated artifact and roadmap

Table 2.2: DSRM phase mapping to project activities and outputs.

2.4 Project evolution timeline

The internship evolved from domain study and exploratory validation to a single-agent operational path, then to the Python reference architecture and evaluation campaigns. Figure 2.4 summarizes this sequence.

The validated workflow established the operational baseline used in the company context. The Python reference architecture should be interpreted as the maintainability-oriented evolution of the same evidence-collection logic, not as a claim that the full operational workflow was replaced during the internship.

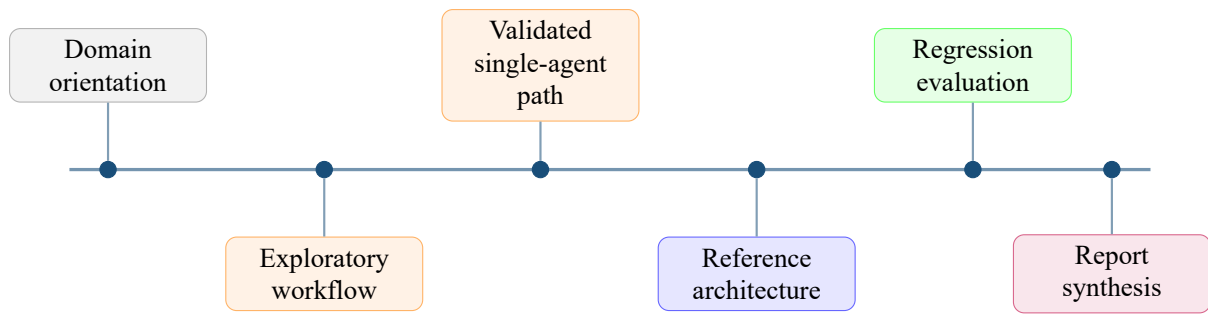


Figure 2.4: Project evolution timeline from domain orientation to evaluated reference architecture.

2.5 Functional and non-functional requirements

The requirements were addressed across two complementary levels. At the operational level, the validated workflow covered the core evidence-collection tasks used in the company context. At the reference-architecture level, the Python implementation generalized those tasks into a more maintainable design with stronger observability, memory, evaluation support, and future scalability.

Table 2.3 summarizes the ten functional requirements derived from the operational context.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	Critical
FR-2	Page-type classification with DOM and screenshot signals	Critical
FR-3	Specialist routing by page type	Critical
FR-4	Tab-aware landing-page candidate discovery	High
FR-5	Multi-server hosting extraction loop	Critical
FR-6	Embedded player activation and stream capture	Critical
FR-7	Hosting-to-embedded fallback	High
FR-8	Screenshot capture and persistent evidence storage	High
FR-9	WHOIS / IP infrastructure enrichment	Medium
FR-10	Operator review surface and trace inspection	High

Table 2.3: Functional requirements derived from the evidence-collection workflow.

Table 2.4 lists the non-functional requirements and the design choices used to address them across the validated workflow and the Python reference architecture.

Taken together, the DSRM framing and the two-level requirement view establish the evaluation scope of the artifact. The implementation-specific constraints, prompt evolution, and agent design principles are presented in Chapter 5 where they directly explain the runtime decisions. The next chapter establishes the technical background needed to understand the website structures, browser tools, streaming protocols, and agentic concepts used by the artifact.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	Validated workflow behavior; Python modular agents, versioned prompts, and tool registry
NFR-2	Observability	Structured event log, evaluation database, SSE review surface, and trace persistence
NFR-3	Cost control	Model routing, provider-side caching, and token budgets
NFR-4	Reliability	Tool error contracts, popup handling, and bounded retry logic
NFR-5	Scalability	Queue-backed worker execution and separable browser services
NFR-6	Portability	Docker Compose, configurable browser profiles, and separable worker services
NFR-7	Evidence traceability	Run-linked artifact chain: event → screenshot → structured output

Table 2.4: Non-functional requirements and implementation responses.

3 Technical Background

This chapter introduces the technical concepts needed before presenting the system architecture and implementation. It deliberately stays at a general level: the goal is to explain the web structures, streaming mechanisms, browser automation tools, and agentic concepts that make the project understandable. The concrete architecture, database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, workflow automation, LangChain, MCP servers, model providers, caching, and token usage.

3.1 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.1.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside an iframe loaded from another domain.

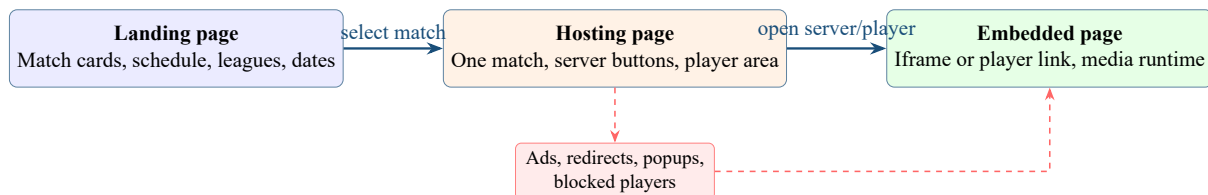


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still, this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.



Figure 3.2: Visual comparison of the three page types used by the extraction workflow: landing discovers candidates, hosting activates and observes the network, and embedded is a fallback player context.

3.1.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids:** repeated blocks with teams, time, league, and a watch link;
- **tables or lists:** one row per event, often with a direct action link;
- **tabbed schedules:** live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections:** match cards that appear only after scrolling or after an XHR request;
- **embedded JSON:** page data stored in `<script type="application/json">` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.1.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as Server 1 or HD, and sometimes channel names or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an iframe or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;
- an iframe whose `src` points to a player page;
- player libraries such as Video.js, JWPlayer, Clappr, hls.js, or dash.js;

- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.1.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside iframes, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any .m3u8 or .mpd URL, while the embedded player requests the manifest after user activation.

Page type	Main role	Useful evidence
landing_page	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
hosting_page	Select server and activate player	server labels, iframe URLs, player state, page screenshot
embedded_page	Load the final media runtime	stream manifest, segment requests, player diagnostics
other	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: Streaming-site page taxonomy and evidence role.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or iframe contains the player? Which runtime request proves that media was loaded?

3.2 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser's tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.2.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

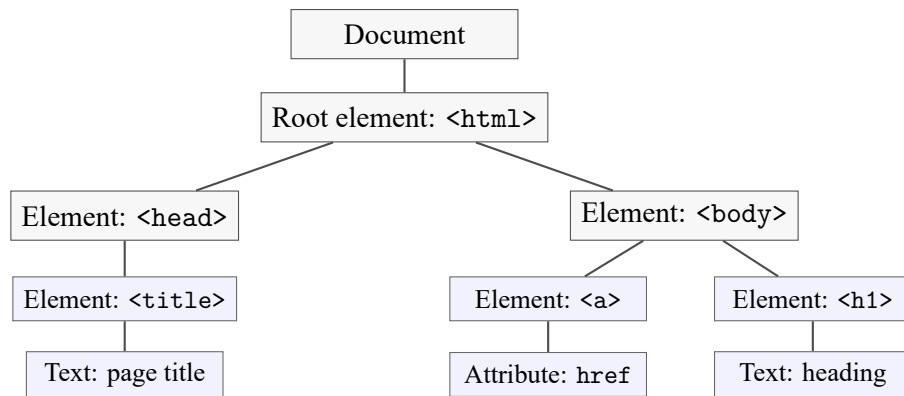


Figure 3.3: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.2.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
<code></code>	landing and hosting pages	provides navigation targets
data-* attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
<code><iframe src="..."></code>	hosting pages	points to embedded player context
<code><video></code> or <code><source></code>	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.3 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.3.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an `iframe` that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

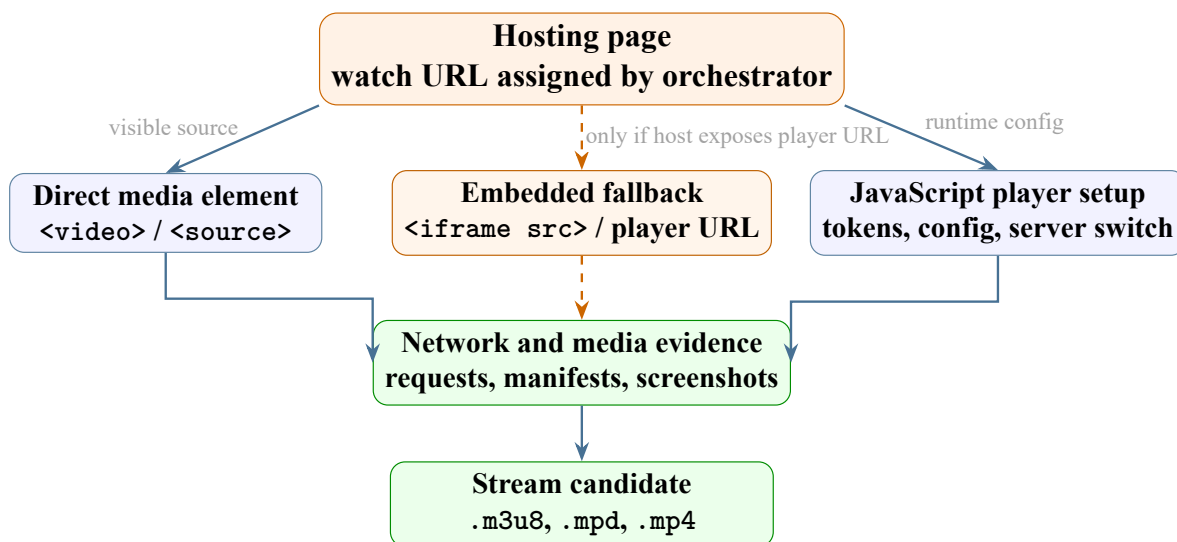


Figure 3.4: Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.3.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a `.m3u8` playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML `.mpd` manifest that describes periods, representations, and segment addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [34, 35].

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

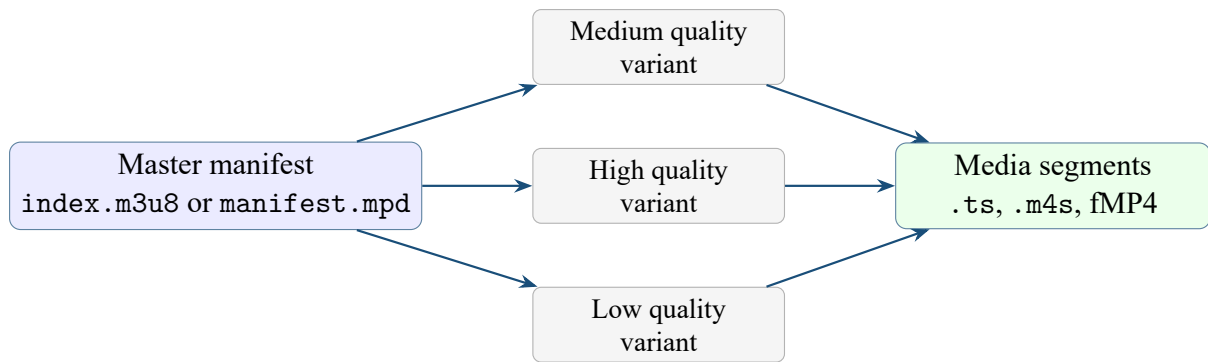


Figure 3.5: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

3.3.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=...` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection [33].

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. The technical background therefore includes both media protocols and web-provider behavior.

3.4 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Browser automation tools such as Puppeteer and Playwright execute pages in real browsers and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity [12, 13, 18].

3.4.1 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only “click a button” but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol;
- read browser-side state with `page.evaluate(...)`.

Puppeteer is therefore the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.4.2 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Browser-level obstacles and handling strategies.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [36]. However, they cannot replace browser reasoning. If a site serves ads and player assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.4.3 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.5 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has

enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.5.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing [1, 3].

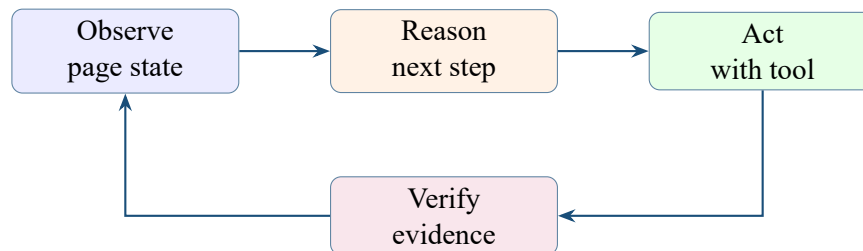


Figure 3.6: General ReAct-style loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.5.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of defining the agent’s role, constraints, tool usage rules, evidence requirements, and output format. In this project domain, a prompt must prevent premature conclusions. It should tell the agent to ground decisions in visible page state, DOM evidence, screenshots, and network observations.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and therefore increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, pricing, and caching [6, 10, 11]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

3.5.3 Workflow automation tools

Workflow automation tools let developers connect nodes for HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow. They are useful for fast validation because a workflow can be assembled and adjusted before a full application is built. In this report, workflow automation is treated as an orchestration concept: it sequences steps, passes data between nodes, calls model providers, and makes evidence-collection logic visible during experimentation. The specific workflow tool used during implementation is introduced in Chapter 5.

3.5.4 LangChain, tools, and MCP servers

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions [24].

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape [26].

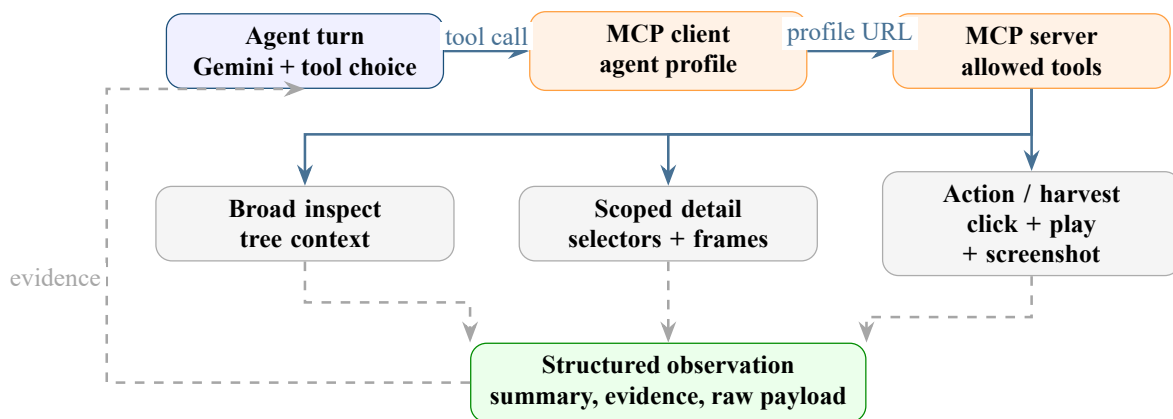


Figure 3.7: General relationship between an LLM agent, profile-scoped MCP tools, and structured browser observations.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.6 Chapter Summary

This chapter established the technical background for the rest of the report. It explained the page flow from landing pages to hosting pages and embedded players, the DOM structures used to inspect those pages, the streaming protocols and player embeds that carry the final evidence, and the browser-level obstacles caused by ads, cookies, popups, iframes, and provider-side checks. It also introduced the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, workflow automation, LangChain, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

4 System Architecture

This chapter presents the reference architecture of the evidence-collection platform from general to specific. It starts with the user-facing system and runtime components, then narrows into the multi-agent pipeline, the handoff contract between agents, the responsibilities of each agent, and the data that makes a run reviewable. Chapter 5 then maps these architectural decisions to the implementation.

Chapter focus

The system is a staged evidence pipeline, not a single unrestricted browser agent. FastAPI starts and records runs. The orchestrator owns the LangGraph route, memory lookup, queues, handoffs, aggregation, provider analysis, and email draft generation. Four browser-facing specialists perform bounded work: classification, landing-page discovery, hosting-page extraction, and embedded-player extraction. Each specialist receives a narrow task, uses only its profile-scoped MCP browser tools, returns typed output, and leaves events that explain both success and failure.

4.1 System Purpose and Architecture Reading Path

The platform is an evidence-collection workbench for illegal streaming websites. The operator launches investigations and reviews the resulting evidence. The engineer or administrator maintains models, prompts, pricing, datasets, browser runtime settings, and MCP tool configuration. External services provide target website content, model inference, screenshot storage, and provider ownership data.

The architecture should be read in four layers:

1. **Product layer:** the operator console launches runs, monitors progress, and reviews evidence.
2. **Runtime layer:** FastAPI validates runtime readiness, schedules jobs, resolves settings, and persists trace data.
3. **Agent layer:** the orchestrator routes work through specialized agents instead of asking one prompt to do every task.
4. **Evidence layer:** streams, screenshots, provider rows, emails, events, model calls, and tool calls are normalized for later review.

4.2 Global Runtime Architecture

The runtime separates the operator console, API, agent execution, browser automation, persistence, and external enrichment services. The Next.js console does not access the database directly; it calls FastAPI endpoints. FastAPI owns background-job execution, active trace as-

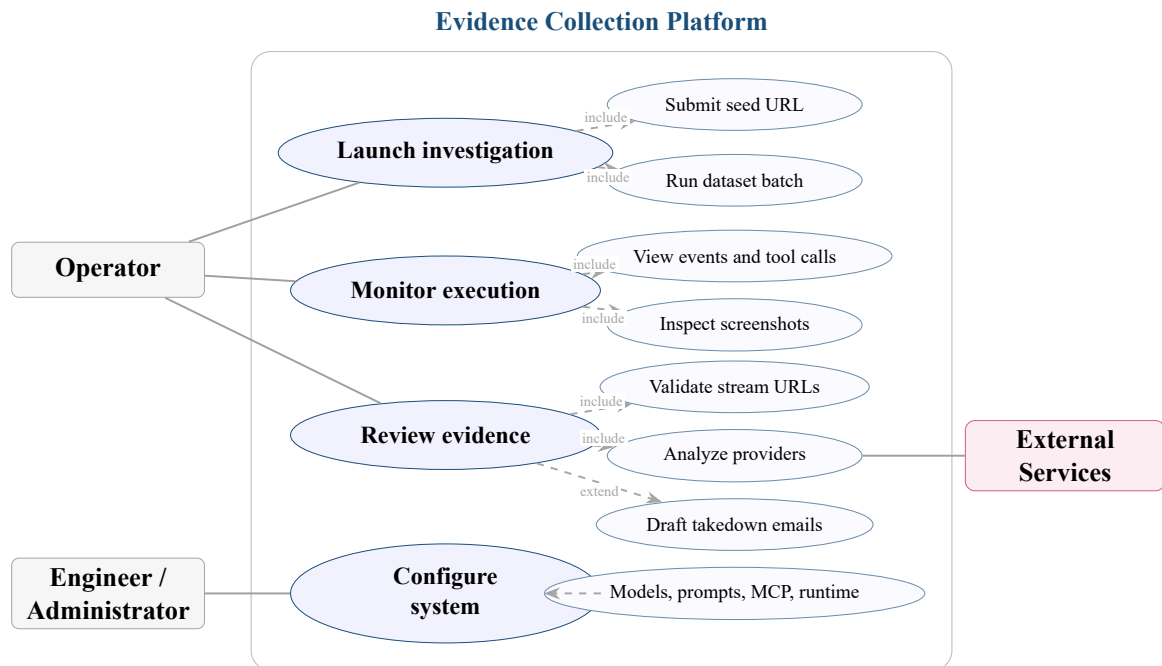


Figure 4.1: Use-case overview of the evidence-collection platform.

Use-case group	User intent	System response
Launch investigation	Submit one URL, a single-agent test, or a dataset batch.	Create a traceable run, validate runtime readiness, and schedule execution.
Monitor execution	Follow agent transitions, tool calls, screenshots, events, and failures.	Stream active state and persist each important runtime event.
Review evidence	Inspect streams, screenshots, provider analysis, metrics, and takedown drafts.	Assemble a reviewable evidence bundle with final status and supporting trace data.
Configure system	Tune models, prompts, MCP tools, browser settings, pricing, and datasets.	Persist configuration and expose health or preflight feedback before runs start.

Table 4.1: Use-case responsibilities across operator and administration workflows.

sembly, settings resolution, and database access. Browser work is delegated to profile-scoped MCP tooling so the agents receive only the tools allowed for their page type.

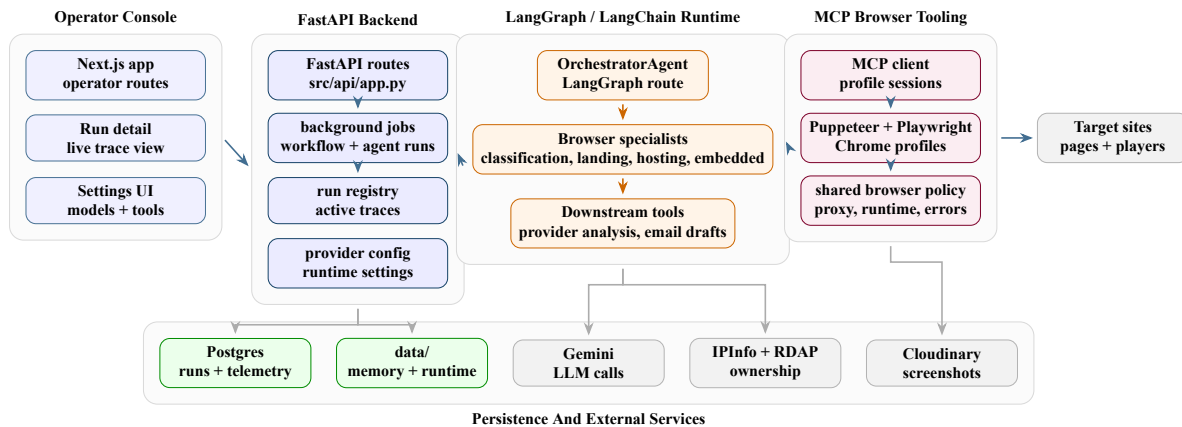


Figure 4.2: Component architecture based on the documented UI, backend, agent runtime, MCP tooling, persistence, and external-service boundaries.

Runtime area	Main responsibility	Why it is separate
Operator console	Launch workflow runs, inspect run detail, manage datasets, providers, tools, browser settings, and model configuration.	The UI stays thin; it consumes API payloads instead of owning persistence.
FastAPI backend	Validate preflight state, create jobs, run background work, assemble active and persisted traces, and expose review payloads.	Runtime control and database access remain testable from backend routes.
Agent runtime	Execute the orchestrator graph and the specialist model/tool loops.	Routing policy remains explicit instead of hidden inside one prompt.
MCP browser tooling	Provide profile-scoped page, frame, screenshot, interaction, and network tools.	Browser dependencies and page failures are isolated from the API process.
Persistence and services	Store normalized run records and call Gemini, Cloudinary, IPInfo, RDAP, and local runtime files.	The final answer and the failure path can be reviewed after execution.

Table 4.2: Global runtime boundaries used by the reference architecture.

This separation is practical rather than cosmetic. Browser automation, LLM reasoning, dashboard rendering, and persistence fail in different ways. Keeping their boundaries visible makes failures easier to classify: a browser-tool timeout, a model retry, a missing stream, a skipped provider lookup, and a dashboard payload issue are different problems.

4.3 End-to-End Agent Pipeline

At large scale, the platform behaves as a controlled multi-agent pipeline, not as one unrestricted browser agent. The operator gives the system a seed URL. The API starts a run, creates an active trace, and calls the orchestrator. The orchestrator checks memory as soft guidance, calls classification, routes to the appropriate specialist path, collects evidence, and only then runs provider analysis and email generation.

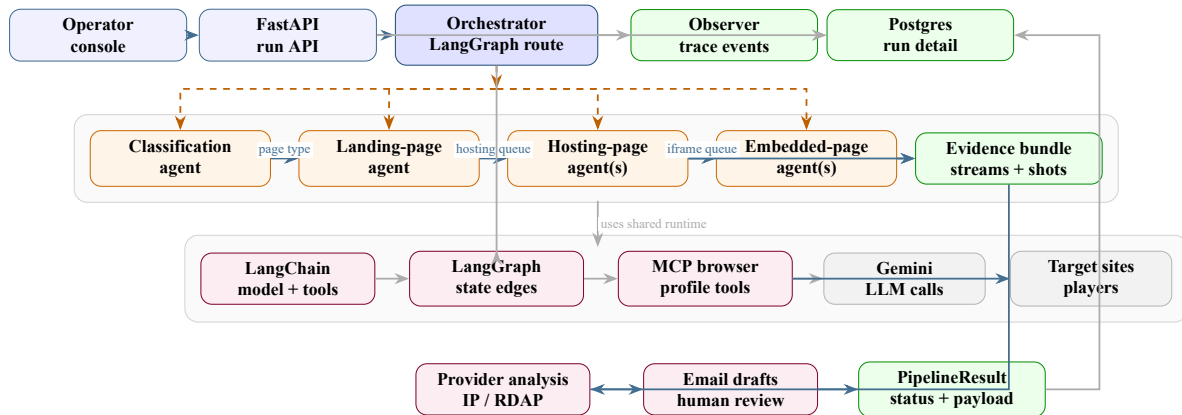


Figure 4.3: End-to-end agent architecture at workflow scale.

The sequence view shows the same workflow as a runtime contract. A run starts as an API request, becomes a background job and active trace, executes through the orchestrator graph, records model and tool activity, and is finally returned to the console as normalized run-detail data.

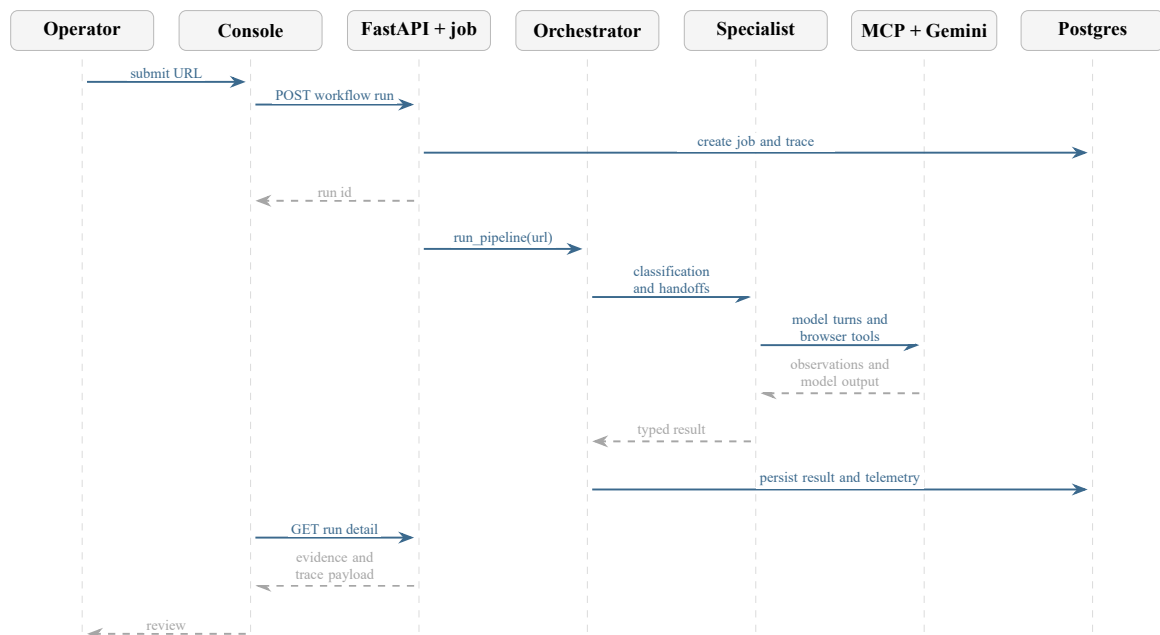


Figure 4.4: Sequence diagram for launching, executing, persisting, and reviewing a workflow run.

The pipeline has three important rules:

- **Agents do not call each other directly:** every specialist returns typed output to the orchestrator.
- **The orchestrator owns queues:** landing can add hosting or embedded targets, hosting can add more hosting or embedded targets, and embedded returns final media evidence.
- **Downstream stages require streams:** provider analysis and email generation are skipped when no stream URL exists.

4.4 Agent Communication and Orchestrator Handoffs

The orchestrator is the routing authority. It builds the LangGraph state machine, checks memory as soft guidance, calls classification, prepares handoffs, executes landing, hosting, and embedded specialists as needed, aggregates structured outputs, then runs provider analysis and email generation only when stream evidence exists. This keeps failures interpretable: a classification timeout, missing hosting candidate, hosting-player failure, embedded-player failure, and provider skip are different operational states.

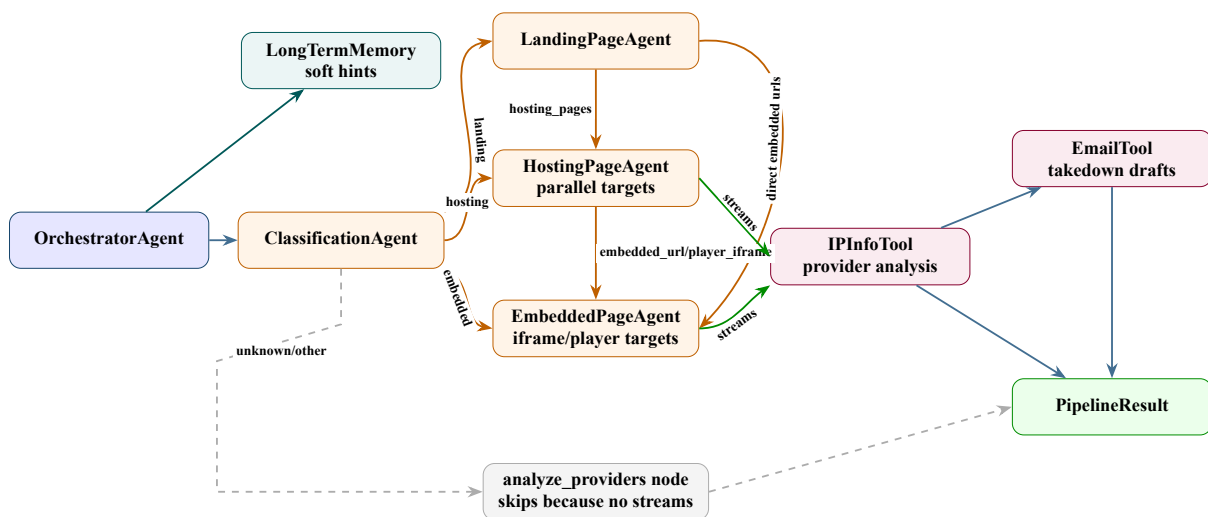


Figure 4.5: Agent interconnection graph based on the architecture documentation.

The handoff is the main communication contract between agents. It contains the root URL, target URL, page type, classification reasoning, route source, memory hints, navigation policy, required evidence, and known pattern context. A specialist receives this context inside its task brief, performs only its bounded job, and returns a schema that the orchestrator can normalize and route.

In this design, communication is visible at three levels. First, route decisions are graph edges, so the next stage is explicit. Second, handoff text gives each specialist the local context it needs without making it own the whole pipeline. Third, the observer records agent starts, model calls, tool calls, screenshots, status changes, costs, and final rollups so the dashboard can explain what happened.

Prompt compilation is also part of the communication contract. Each browser-facing specialist receives the same architectural ingredients but a different stage contract. The compiled

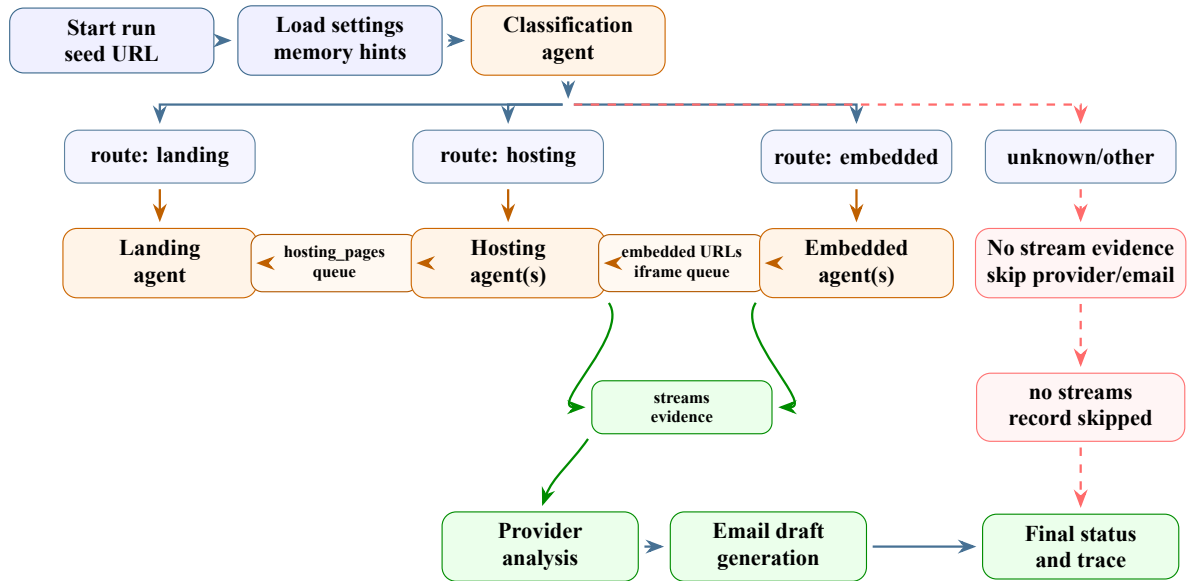


Figure 4.6: Orchestrator routing flowchart and handoff decisions.

Stage	Main input	Responsibility	Main output
Orchestrator	Seed URL, settings, observer, and memory hints.	Build the LangGraph route, prepare handoffs, aggregate results, compute final status, and call downstream tools.	PipelineResult, trace events, provider rows, and email drafts.
Classification	URL and profile-scoped browser context tools.	Decide page type without extracting streams or provider data.	Classification result with page type, confidence, and reasoning.
Landing	Landing URL plus orchestrator handoff.	Discover watch or hosting candidates while preserving match, iframe, channel, and player hints.	ExtractionResult with match records and hosting or embedded candidates.
Hosting	Assigned hosting URL plus evidence checklist and navigation policy.	Operate server/source controls, activate playback, collect network evidence, and return explicit embedded handoffs when needed.	Screenshots, server rows, stream URLs, diagnostics, or embedded player URLs.
Embedded	Verified player or iframe URL plus handoff context.	Stay inside the assigned player context, recover from drift, and harvest final stream evidence.	Stream URLs, screenshots, frame diagnostics, and player state.
Provider and email tools	Stream URLs, provider rows, screenshots, and extractions.	Enrich stream hosts and draft reviewable takedown notices.	ProviderInfo records and TakedownEmail drafts.

Table 4.3: Agent and tool responsibility map.

prompt combines the base policy, the agent-specific contract, the URL task brief, runtime context, long-memory hints, and short-memory working state. Short memory is run-local: it tracks recent navigation, tool calls, selectors, candidates, streams, server records, screenshots, and the next working objective. Long memory is cross-run site memory: it stores summarized playbooks in the site-memory database and profile JSON so a future run can receive hints without trusting them blindly. Both are soft context; live browser evidence still decides the route.

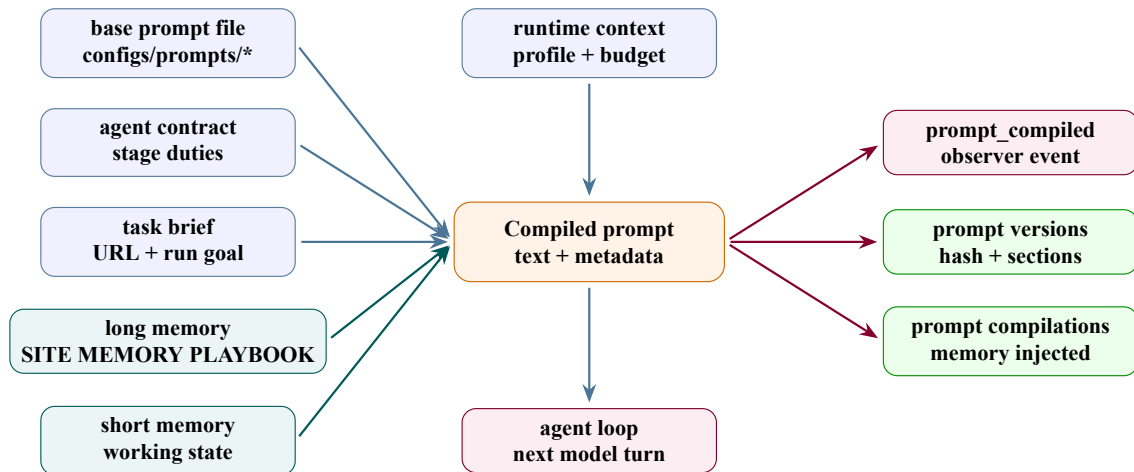


Figure 4.7: Prompt compilation and memory inputs for each browser-facing specialist.

4.5 Agent-by-Agent Design

4.5.1 Orchestrator Agent

The orchestrator is the control plane for one run. It owns state, queues, handoffs, aggregation, provider and email execution, final status, and trace emission, but it does not inspect pages directly.

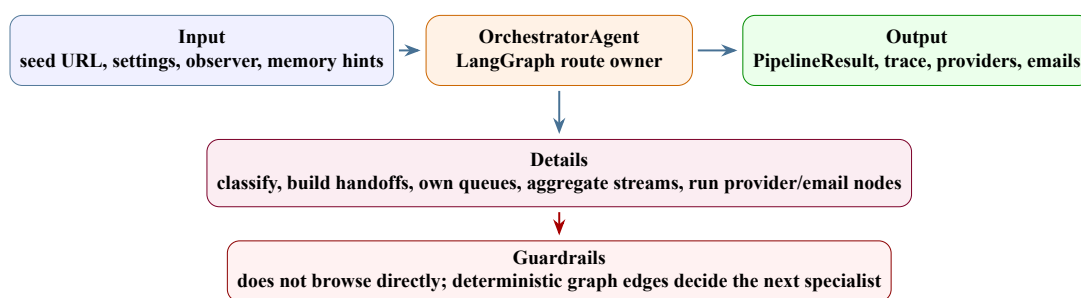


Figure 4.8: Orchestrator-agent design card.

4.5.2 Classification Agent

The classification agent answers one question: what type of page is this URL? It distinguishes landing, hosting, embedded, unknown, and irrelevant pages before extraction begins.

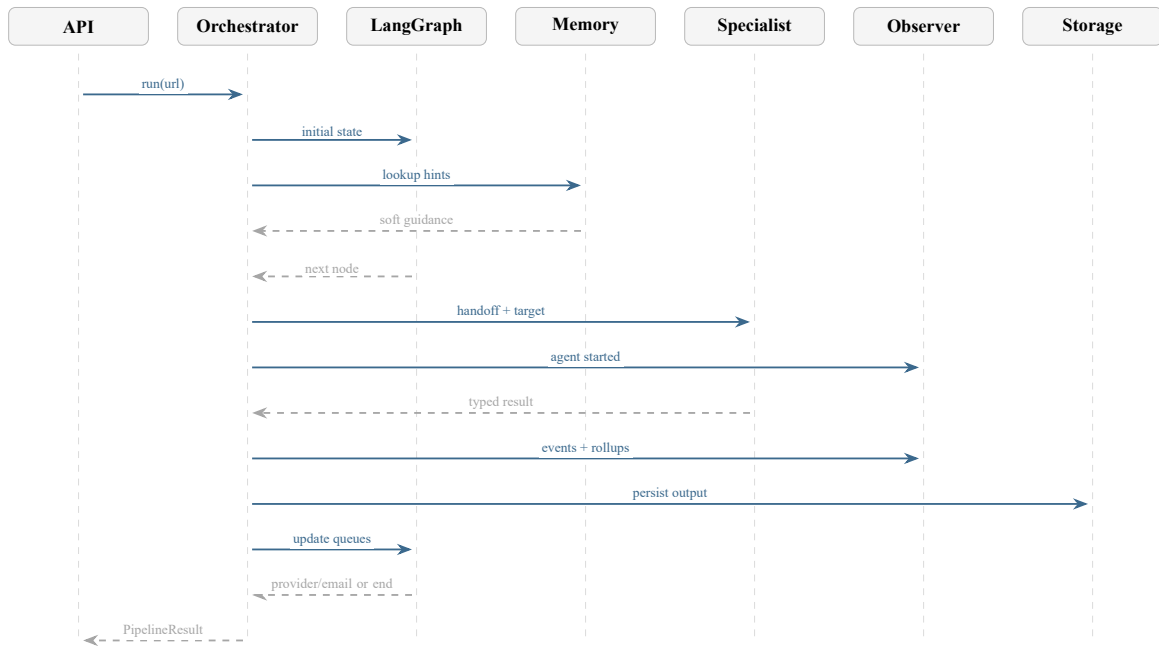


Figure 4.9: Sequence diagram for orchestrator handoff, trace recording, and route updates.

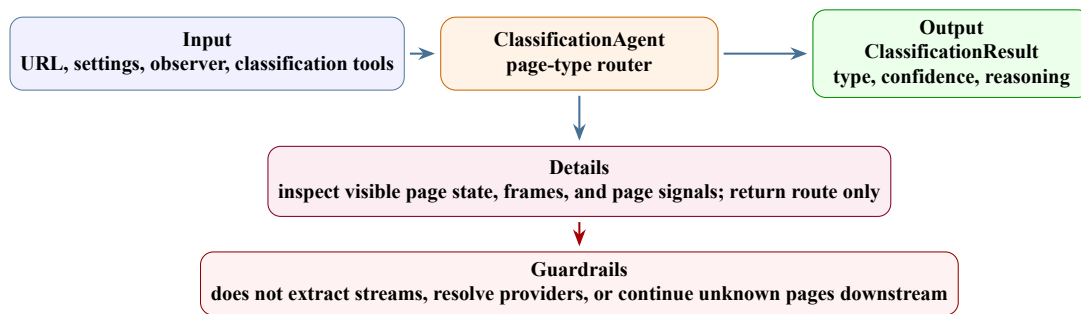


Figure 4.10: Classification-agent design card.

A landing result sends the run to the landing agent. A hosting result queues the root URL for the hosting agent. An embedded result queues the root URL for the embedded agent only when the page looks like a direct player. Unknown or irrelevant pages skip provider and email stages because no stream evidence exists.

4.5.3 Landing-Page Agent

The landing-page agent turns broad site pages into bounded downstream targets. It discovers useful watch, hosting, embedded, or direct-stream candidates from homepages, schedule pages, directory pages, and match listings.

The orchestrator deduplicates landing outputs. Normal watch or server pages go to the hosting queue, direct player URLs go to the embedded queue, and provider-like media URLs are preserved as stream evidence.

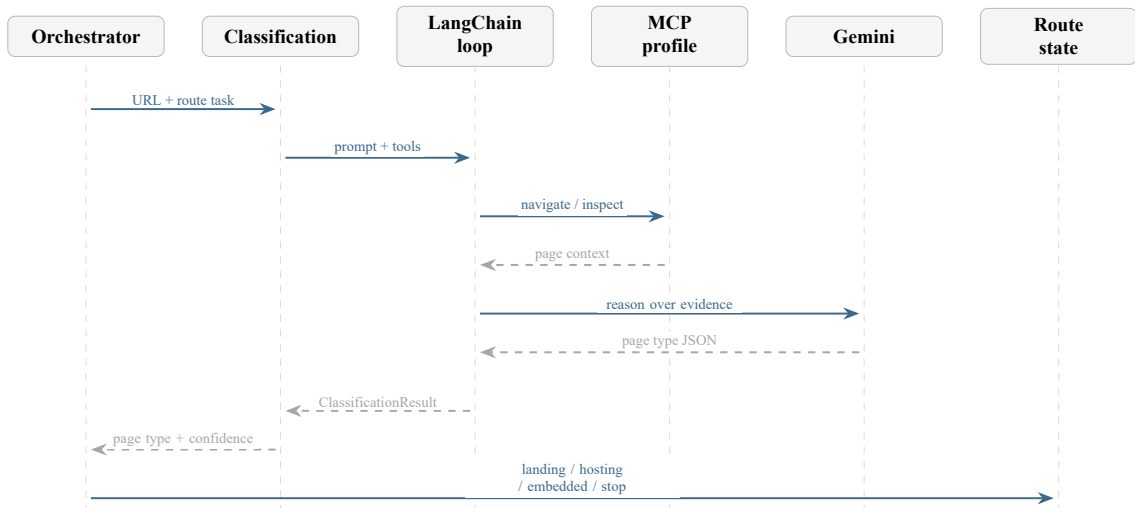


Figure 4.11: Classification-agent sequence: inspect page state and return a routing decision.

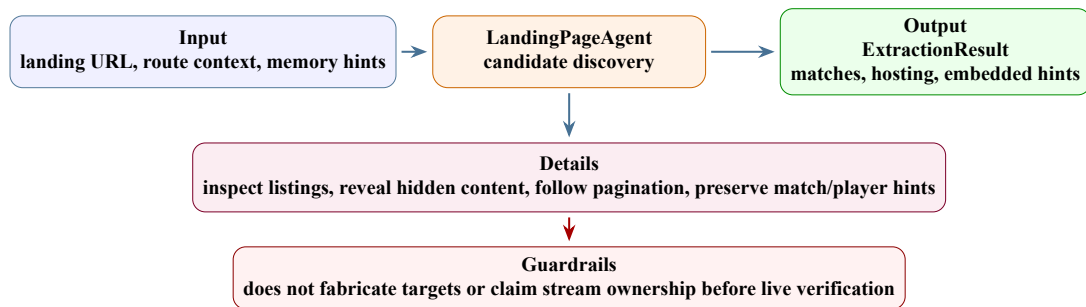


Figure 4.12: Landing-page-agent design card.

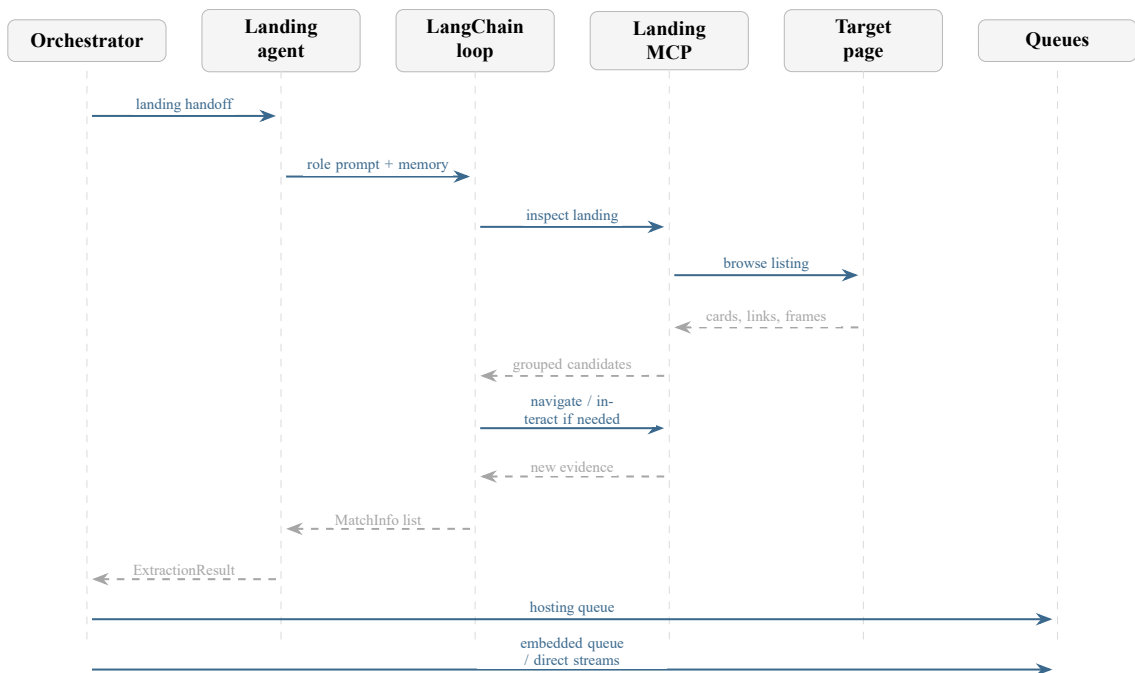


Figure 4.13: Landing-agent sequence: convert listing-page evidence into downstream queues.

4.5.4 Hosting-Page Agent

The hosting-page agent owns watch pages and server/source selection pages. It activates players, operates server controls, handles overlays, captures screenshots, and returns streams or explicit embedded-player handoffs.

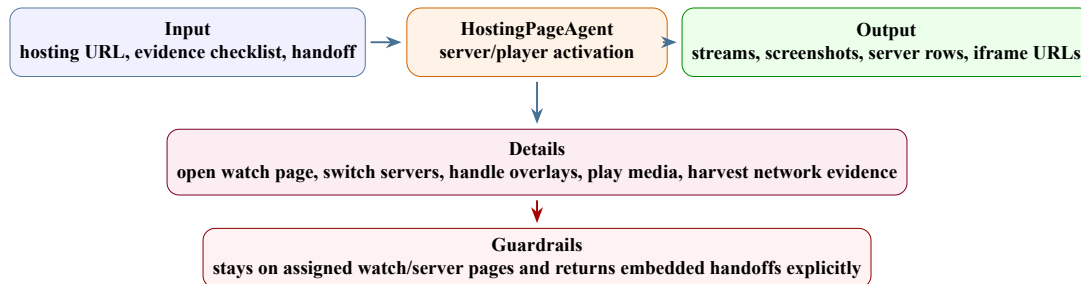


Figure 4.14: Hosting-page-agent design card.

If the page exposes an iframe or direct player URL instead of a stream, the hosting result carries that URL as an embedded handoff. If playback fails, it still returns diagnostics so the run explains why extraction stopped.

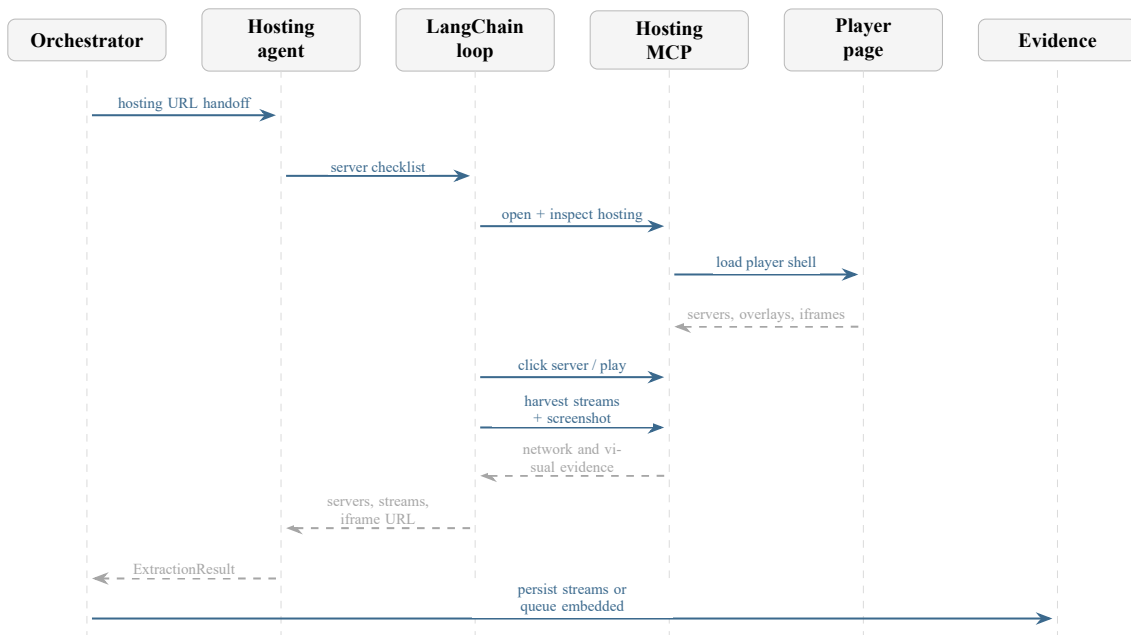


Figure 4.15: Hosting-agent sequence: activate watch pages and return stream or iframe evidence.

4.5.5 Embedded-Page Agent

The embedded-page agent owns direct player or iframe URLs. It stays inside the assigned player context, inspects frames, activates iframe-local controls, recovers from popup or redirect drift, and collects final media evidence.

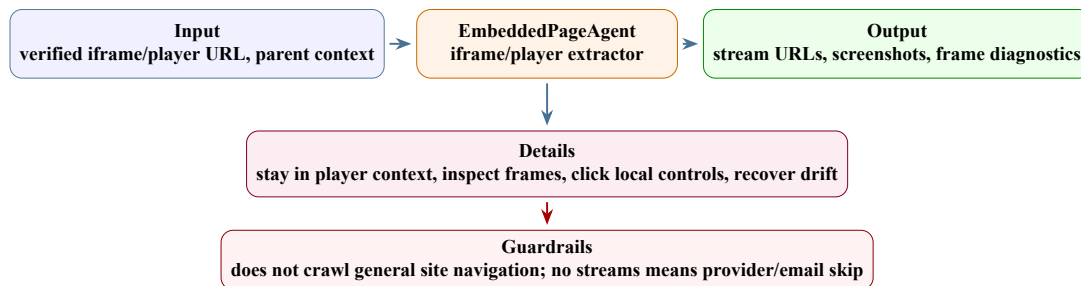


Figure 4.16: Embedded-page-agent design card.

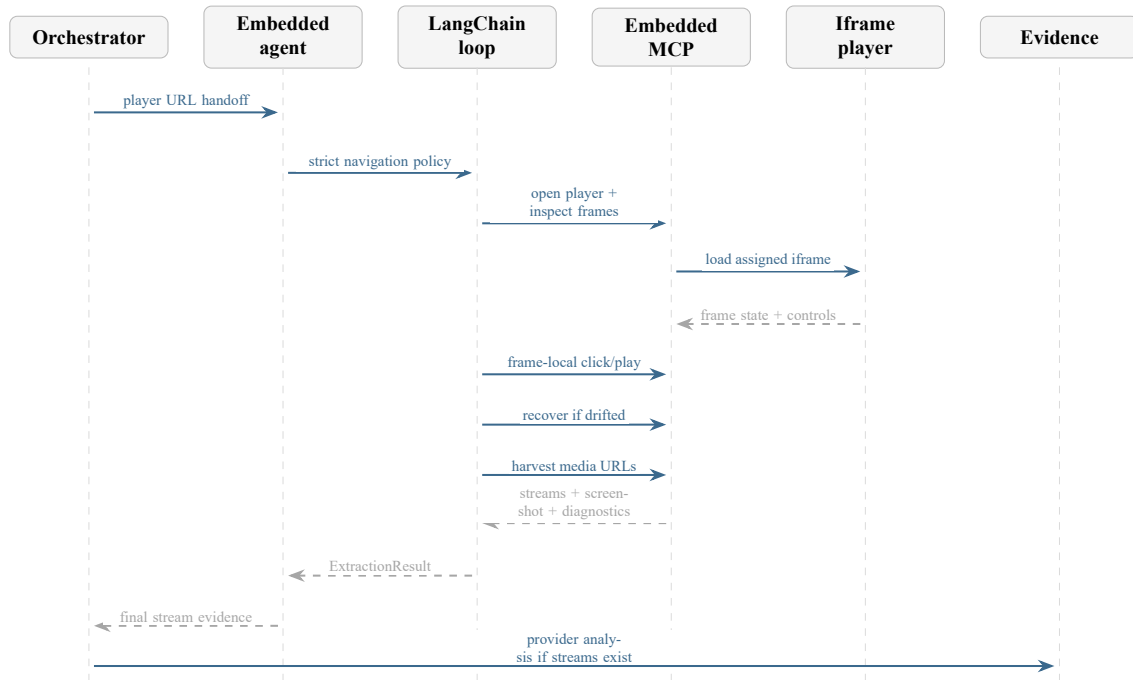


Figure 4.17: Embedded-agent sequence: stay in the player context and harvest final media evidence.

4.5.6 Provider Analysis and Email Generation

Provider analysis and email generation are downstream evidence-packaging stages. They are not page-navigation specialists. Provider analysis reads the collected stream URLs, resolves host or IP ownership, and returns provider information and abuse contact data. Email generation groups provider rows and stream evidence into reviewable takedown drafts. These drafts are not sent automatically; they are artifacts for human review.

4.6 Data, Trace, and Reviewability View

The reviewability view is intentionally compact. It follows the runtime input and output graph from the agent documentation rather than reproducing the full database schema or every helper function. Settings, the observer, the orchestrator handoff, and the target URL enter each browser-facing specialist. The specialist then compiles prompt metadata, receives only profile-scoped MCP tools, and returns a Pydantic result. The observer and the typed result both feed the normalized run tables used by the run-detail console.

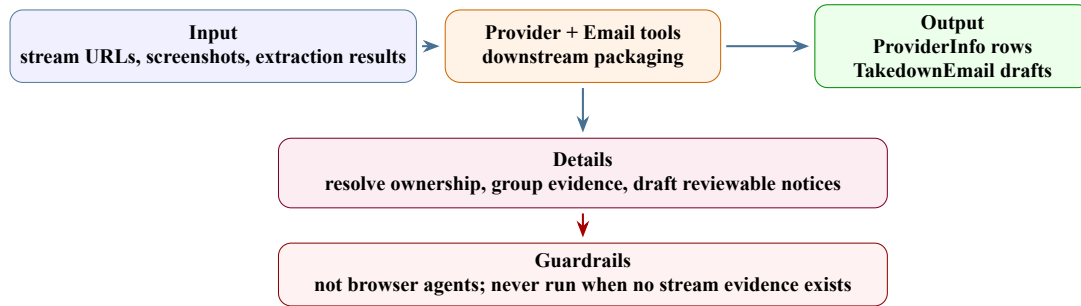


Figure 4.18: Provider-analysis and email-generation design card.

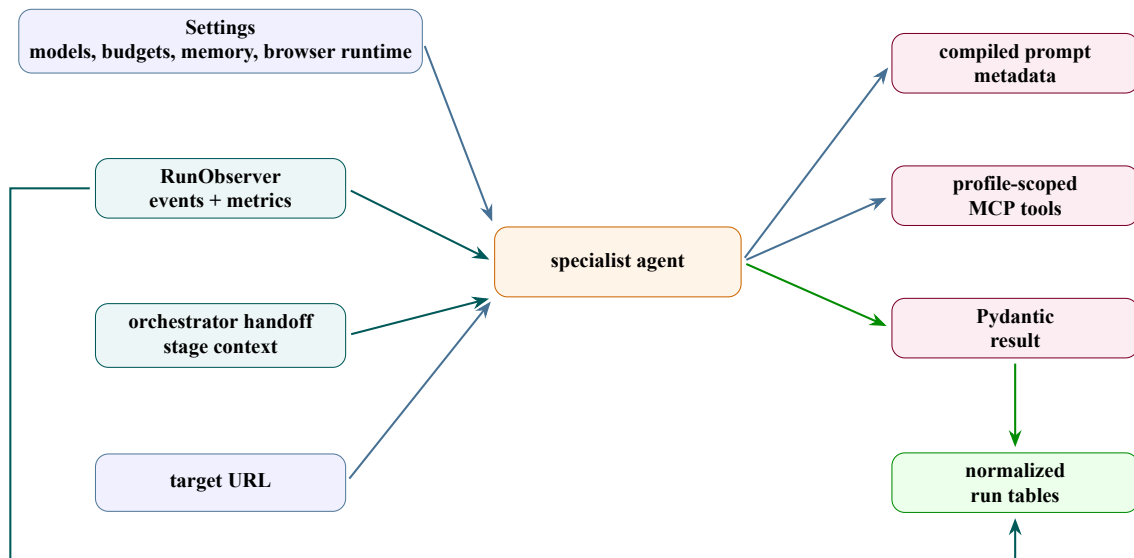


Figure 4.19: Runtime inputs and outputs for browser-facing agents based on the agent documentation.

Architectural element	Boundary	Reviewable output
Orchestrator state	Current URL, classification, pending hosting and embedded queues, extraction results, provider rows, emails, and final error state.	Route decisions and final PipelineResult.
Handoff context	Root URL, target URL, page type, route source, evidence checklist, navigation policy, and memory hints.	Bounded specialist task context.
Runtime observer	Agent lifecycle, model calls, tool calls, screenshots, status changes, cancellation, and cost counters.	Events, rollups, traces, and metrics.
Persistence layer	Background jobs, run records, agent runs, evidence rows, screenshots, streams, provider rows, and email drafts.	The data behind GET /ui/runs/{run_id}.

Table 4.4: Runtime data boundaries and persisted review outputs.

4.7 Chapter Summary

The architecture is a staged, reviewable evidence pipeline. The operator starts and reviews runs through the console. FastAPI controls execution and persistence. The orchestrator uses LangGraph to route between agents and to preserve handoff state [24, 25]. Each specialist agent uses LangChain and LangGraph for the model and tool loop, while MCP browser tools gather page, frame, screenshot, and network evidence. Provider and email tools run only after streams exist, and storage preserves both the final answer and the failure path. The next chapter maps these architecture boundaries to the concrete implementation.

5 Implementation

The implementation progressed in three major parts. The first part was an n8n workflow used as a fast exploration and validation environment. The second part was a Python reference solution built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability. The third part was the deployment direction: the company preferred to keep n8n as the operational workflow, while one agent was deployed as a focused cloud job.

This split matters because the two systems did not have the same purpose. The n8n workflow was useful for testing whether agentic browsing could handle the domain at all. The Python implementation was built after those lessons, with cleaner engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation. The deployment work then narrowed the cloud target to the part that the company actually wanted to operationalize during the internship.

5.1 Part 1: n8n Workflow Prototype

5.1.1 Purpose of the n8n phase

The n8n implementation acted as the project playground [27]. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

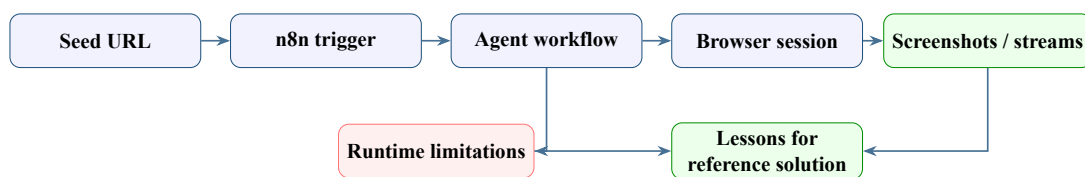


Figure 5.1: Role of the n8n workflow as a feasibility playground.

The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

5.1.2 n8n workflow screenshots

The n8n workflow used agent-like nodes to test the same page-type responsibilities that later became specialist agents. Figure 5.2 shows the full sequential workflow and the four agent workflow captures used during the prototype phase.

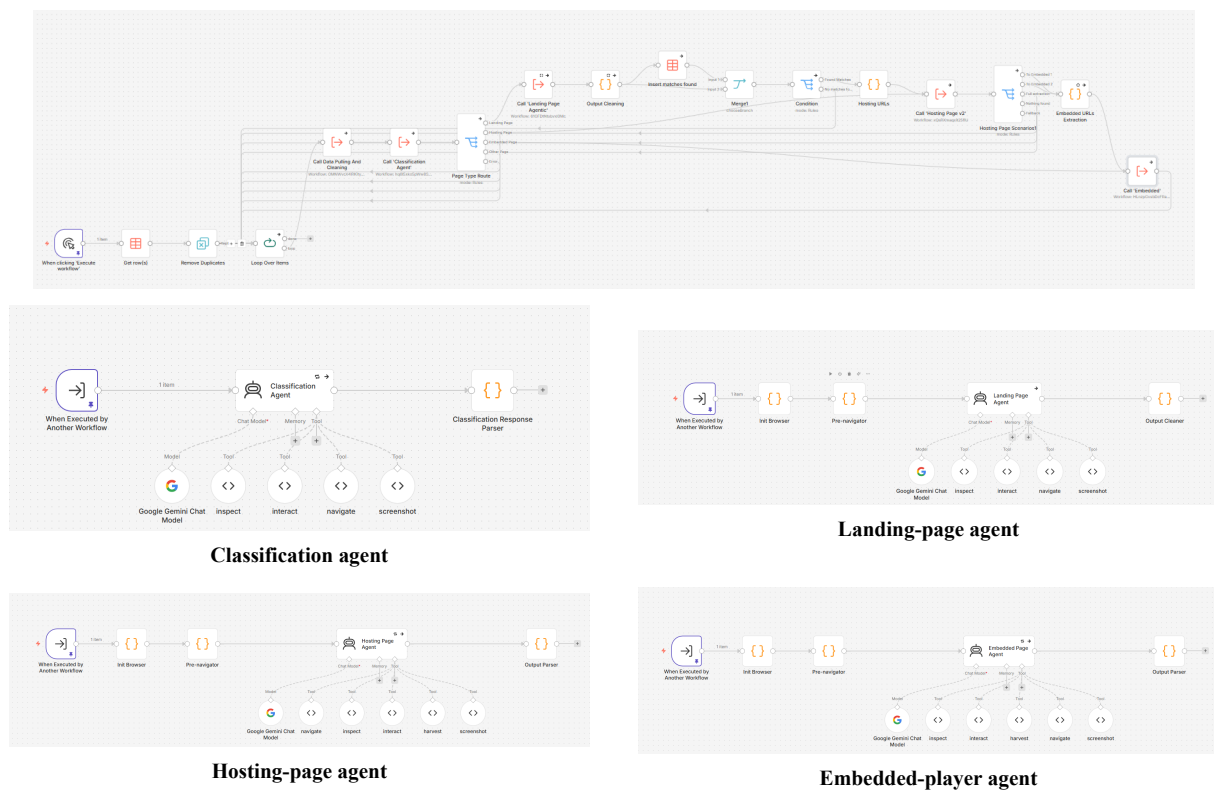


Figure 5.2: n8n workflow screenshots: full sequential workflow and the four specialist agent workflows.

n8n agent role	What it tested	Lesson learned
Classification	Whether a page could be categorized as landing, hosting, embedded, or other from browser evidence.	Routing must be explicit and fail-closed; wrong classification wastes the whole run.
Landing	Whether listing pages could expose match cards, tabs, and hosting links.	Generic DOM inspection was not enough; landing pages needed page-type-specific extraction.
Hosting	Whether server buttons and play controls could be activated.	Stream evidence appears only after interaction; screenshot and media-state validation are necessary.
Embedded	Whether iframe and player pages could reveal final manifests.	Nested frames require a dedicated extraction path and should not be mixed with landing-page exploration.

Table 5.1: n8n prototype agent roles and implementation lessons.

5.1.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Sequential execution	The workflow behaved mostly as a sequential chain rather than parallel specialist agents.	Runs were slower than necessary, especially when multiple hosting or embedded candidates existed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Workflows had to be exported as JSON files to preserve progress and compare versions.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A <code>puppeteer.connect</code> workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.2: n8n prototype limitations and engineering consequences.

5.2 Part 2: Implemented Reference Solution

The reference solution keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

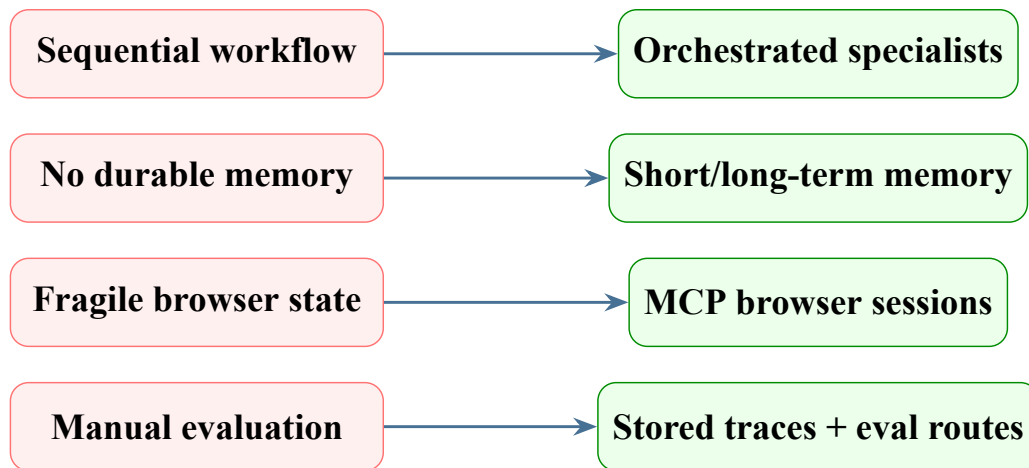


Figure 5.3: How n8n limitations shaped the reference implementation.

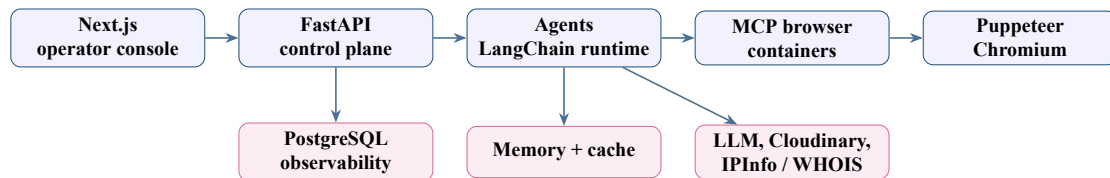


Figure 5.4: Five implementation areas of the reference solution.

5.2.1 FastAPI Backend

The FastAPI backend is the control plane of the reference solution [22]. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
MCP tool diagnostics	Expose profile-specific tool status and logged tool calls.	Separates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.3: Backend API responsibilities in the reference implementation.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

5.2.2 Next.js Operator Console

The Next.js frontend is not just a dashboard [23]. It is the operator console for the reference implementation: launching runs, following execution, reviewing evidence, and changing runtime configuration.

Frontend surface	What it shows or controls	Main backend dependency
Dashboard	Overview, cost, token, provider, tool, and agent telemetry.	Dashboard aggregation endpoints and persisted observability tables.
Live Pipeline	Workflow and single-agent launch with runtime preflight.	Workflow launch, health, and browser/MCP readiness endpoints.
View Results	Website dataset, batch execution, per-site status, and run history.	Dataset, batch, and run repositories.
Provider Results	Stream URL enrichment, IP resolution, provider distribution, country distribution, and manual lookup.	Provider-analysis records and lookup services.
Settings	Gemini model assignments, cache/runtime controls, browser settings, display preferences, API key status, notifications, and MCP tool enablement.	Configuration endpoints and runtime settings persistence.

Table 5.4: Current Next.js operator-console surfaces and backend dependencies.

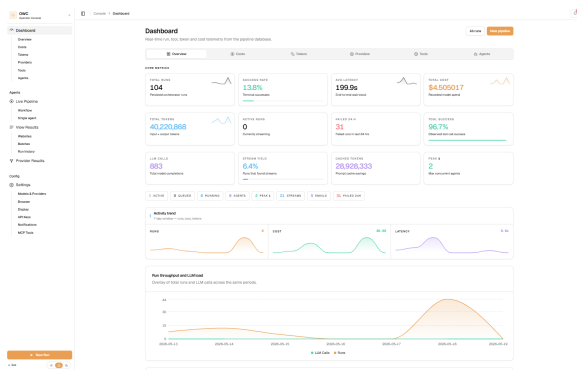
The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found, and where the model or browser runtime failed.

5.2.3 MCP Browser Containers and Puppeteer Tools

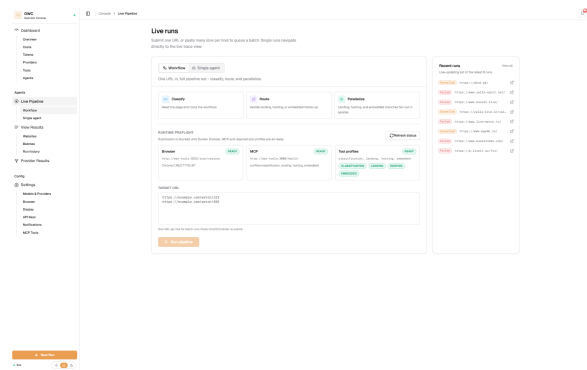
The browser tooling was extracted from the n8n code-node style into dedicated MCP browser services [26]. Each agent profile receives a filtered tool surface through an MCP endpoint. This reduces tool-selection noise and keeps browser session ownership outside the LLM prompt. The engine comparison below is based on the official Playwright and Puppeteer documentation and the project’s integration constraints [13, 12, 18].

5.2.4 Agents, Models, Memory, and Prompts

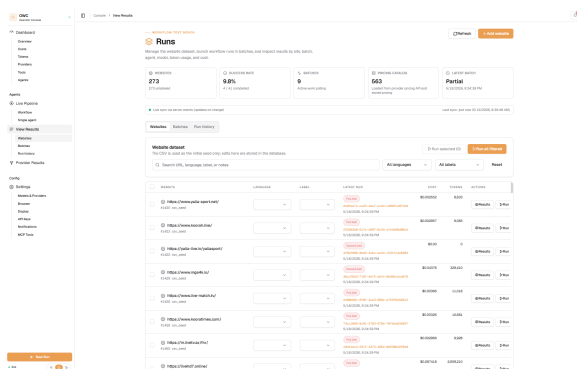
The reference solution uses specialist agents rather than one general browser agent. Each specialist receives a role-specific prompt, a profile-specific tool set, and a bounded execution budget.



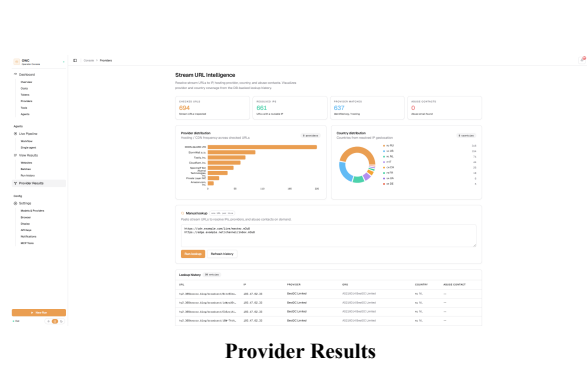
Dashboard telemetry



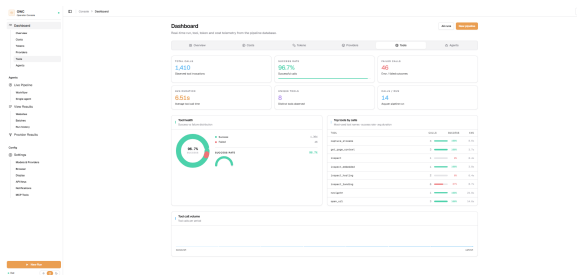
Live Pipeline launcher



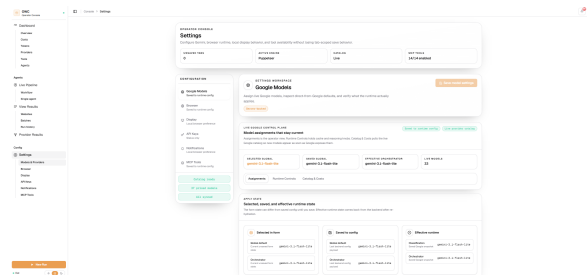
View Results dataset



Provider Results



Tool telemetry



Model settings

Figure 5.5: Current Next.js operator-console screens used for launch, review, telemetry, provider enrichment, and runtime configuration.

Browser engine	Strengths observed	Project decision
Playwright	Strong context isolation, good frame and browser-context APIs, useful for comparing behavior and validating browser policies.	Tested and kept as a useful reference path, but not the primary runtime.
Puppeteer	Direct Chrome DevTools Protocol fit, simpler integration with the existing tool registry, stable enough for the profiled MCP tool server.	Chosen as the main browser engine for the implemented agent prompts and tool runtime.

Table 5.5: Browser automation engine comparison for the MCP tool layer.

Tool group	Purpose
Navigation tools	Open URLs, go back, wait for page state, and recover from navigation drift.
Inspection tools	Read page context, landing cards, hosting server controls, embedded player state, DOM elements, and frame trees.
Interaction tools	Click by selector, text, XPath, checkbox/radio/select, coordinates, play media, type input, scroll, and swipe.
Evidence tools	Capture screenshots, media state, stream requests, player signals, and harvested m3u8/mpd/mp4 candidates.
Memory tools	Read or update reusable site hints from the memory layer.

Table 5.6: Puppeteer MCP tool groups and responsibilities.

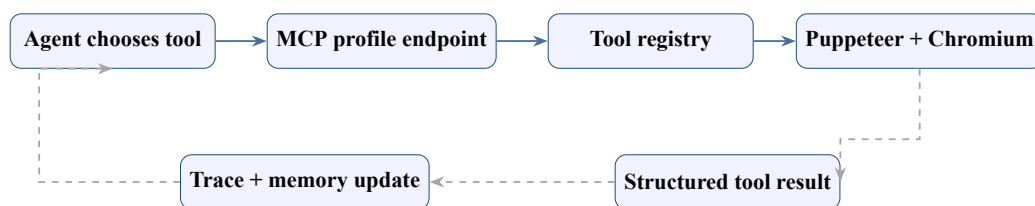


Figure 5.6: Browser tool execution loop through MCP and Puppeteer.

Agent	Objective	Main tools used
Classification	Decide whether the seed is landing, hosting, embedded, or other.	Generic inspection, screenshot, frame tree, memory lookup.
Landing	Discover match cards, listing tabs, pagination, and hosting candidates.	Landing inspect, element queries, click/scroll, screenshot, memory tools.
Hosting	Activate server controls, verify player state, and extract streams or embedded fallbacks.	Hosting inspect, click/play tools, media state, frame tree, harvest, screenshot.
Embedded	Work only on embedded/player URLs and harvest final stream evidence.	Embedded inspect, play/media tools, harvest, screenshot, frame diagnostics.

Table 5.7: Specialist agent objectives and assigned tool surfaces.

5.2.4.1 Design constraints implemented in the agent runtime

The agent runtime was shaped by constraints that became clear during prototyping. These constraints are implementation concerns, not only methodology notes, because each one changed how the agents, prompts, tools, memory, and stopping rules were built.

5.2.4.2 Prompt evolution in the implementation

The prompts evolved from linear intent prompts into evidence-driven loops. The early prompt asked the model to solve the page in one pass. That worked on simple pages, but it failed on ambiguous layouts because the agent would stop before opening tabs, activating

Constraint	Runtime effect	Implementation response
Changing layouts	Selectors and visible labels differ across languages, clones, and match-list layouts.	Agents use broad DOM context first, then scoped element details and page-type-specific tools.
Nested iframe players	The playable media source may not exist in the top-level page.	Hosting agents can hand off embedded URLs; embedded agents inspect frame trees and player-only pages.
Popups and ad traps	Clicks can open new tabs, steal focus, or hide the original player.	Browser tools restore focus, close popup tabs, and retry with safer JavaScript interactions.
Delayed stream generation	Stream manifests often appear only after a play click or server switch.	Prompts require media-state or screenshot verification before harvesting network evidence.
Cost and context growth	Repeated DOM observations, screenshots, and retries can make runs expensive.	The runtime tracks token usage, uses caching, and enforces bounded tool budgets.
Evidence isolation	The reference system must not depend on company production data.	Persistence is limited to traces, metrics, screenshots, tool calls, model usage, and structured outputs.

Table 5.8: Runtime constraints and their implementation responses.

players, or verifying stream generation. The implemented prompt pattern instead forces each specialist to observe, reason, act, and validate before returning a result.

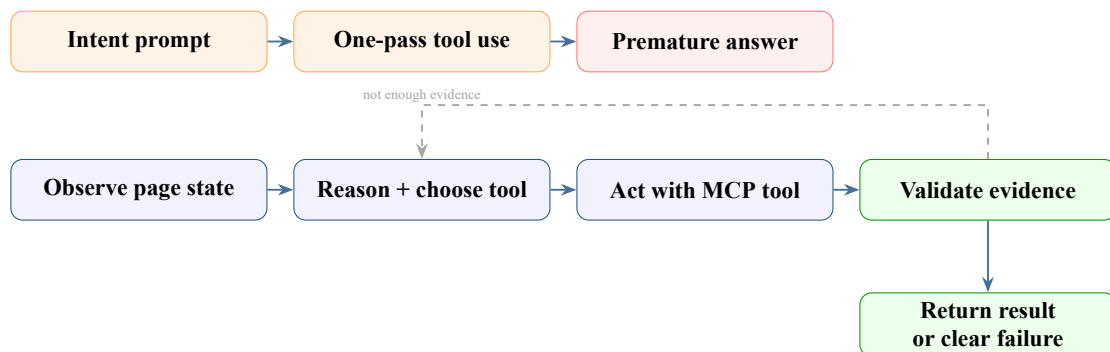


Figure 5.7: Prompt evolution from one-pass intent prompts to evidence-driven agent loops.

5.2.4.3 Agent design principles

The resulting agent design follows seven practical principles:

- classify the page role before deep extraction;
- separate operational n8n validation from the Python reference architecture;
- ground every conclusion in browser evidence rather than textual guessing;
- preserve traces, screenshots, and structured outputs for each critical step;

- use specialist agents and explicit fallbacks instead of one unconstrained agent;
- optimize reliability and cost together through caching, budgets, and model routing;
- keep deployment loosely coupled across API services, browser workers, observability storage, and queue-based execution.

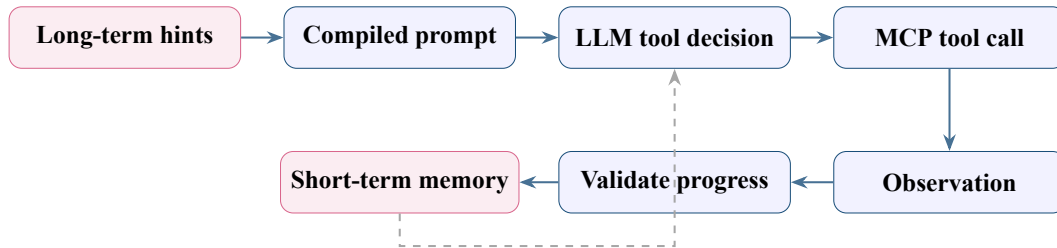


Figure 5.8: Agent reasoning loop with memory and tool feedback.

LangChain is used as the tool-binding and message-loop layer [24]. The implementation does not let the model freely access every browser capability. Instead, the MCP client loads only the tool profile for the current agent. The shared loop records tool calls, model calls, token usage, cache behavior, and stop reasons.

5.2.4.4 Model behavior, token usage, and rate limits

The model layer had to be treated as an operational dependency, not only as a prompting detail. Long browser pages, repeated DOM observations, screenshots, tool traces, and failed retries can grow the context quickly. The implementation therefore tracks context windows, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are evaluated across RPM, TPM, and RPD. Google’s current documentation defines these as requests per minute, input tokens per minute, and requests per day, and states that limits depend on model and usage tier and can change in AI Studio [9]. The implementation therefore treats rate limits as configuration and observability concerns rather than hard-coded constants.

5.2.4.5 Prompt engineering and stopping behavior

The prompts evolved from broad goals into operational procedures. Each prompt defines the objective, the evidence required, which page ownership rules must be respected, and when the agent should stop. The strongest prompt pattern was an evidence-based loop: observe, state the current hypothesis, choose one action, verify the result, then either continue or return a structured failure.

5.2.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its purpose in this project is to preserve what happened during a run so that failures and successes can be inspected later.

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.9: Model-runtime concerns handled by the agent platform.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.10: Prompt engineering rules enforced by the specialist agents.

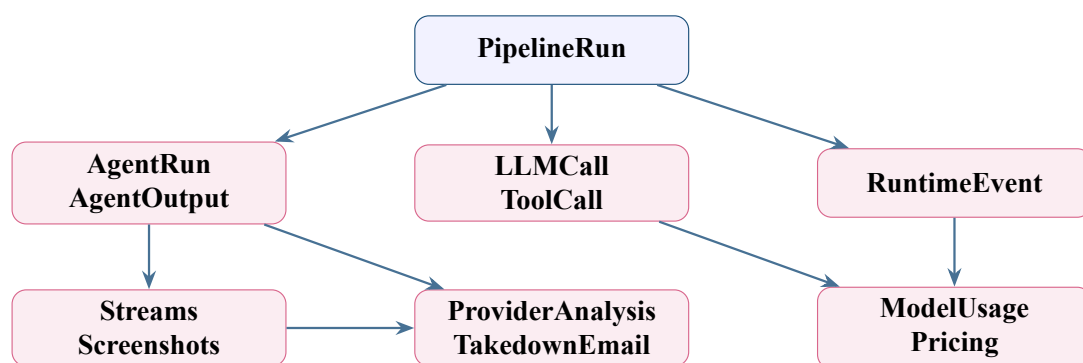


Figure 5.9: Simplified database relationship map for run review.

Record group	Tables / records	Status in the project
Core run trace	Pipeline runs, agent runs, runtime events, LLM calls, tool calls.	Central to reviewing and debugging runs.
Evidence	Run streams, screenshots, provider analyses, takedown emails.	Central to the evidence bundle.
Model and prompt metadata	Prompt versions, prompt compilations, model usage, pricing configs.	Important for cost, prompt iteration, and reproducibility.
Memory	Memory entries and memory hints used.	Used to connect cross-run knowledge with current agent decisions.
Datasets and batches	Dataset sites, dataset batches, dataset site runs.	Useful for evaluation/batch execution, but not always used in every run.
Operational support	Run decisions, run tasks, tool-call diagnostics, snapshots.	Support/diagnostic records; some remain secondary rather than core report outputs.

Table 5.11: PostgreSQL observability records and their implementation role.

5.3 Part 3: Deployment

The company preferred to keep n8n as the main operational workflow. For that reason, the cloud deployment did not attempt to replace the whole workflow with the Python multi-agent platform. Instead, the deployment focused on one validated agent: the landing page agent. The deployment shape follows the same practical pattern already used for n8n: one containerized unit owns the runtime it needs. For the landing-agent job, this means the Azure Container Apps Job image contains the Python worker, the Puppeteer MCP server, and Chrome in the same container. Azure Service Bus [28, 29] passes a landing-page job message; the container job wakes up, runs the browser-agent logic through its local MCP endpoint, persists logs and evidence, and returns extracted match and host candidate evidence.

This deployment choice matches the internship scope. The Python reference architecture shows how the full platform can be structured, but the company-side deployment remained focused: n8n orchestration stayed in place, while the landing-page agent was isolated as a cloud-executable single-container unit.

5.4 Implementation Summary

The implementation moved from a useful but limited n8n prototype to a structured reference solution. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. The Python solution addressed those problems with explicit back-end routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

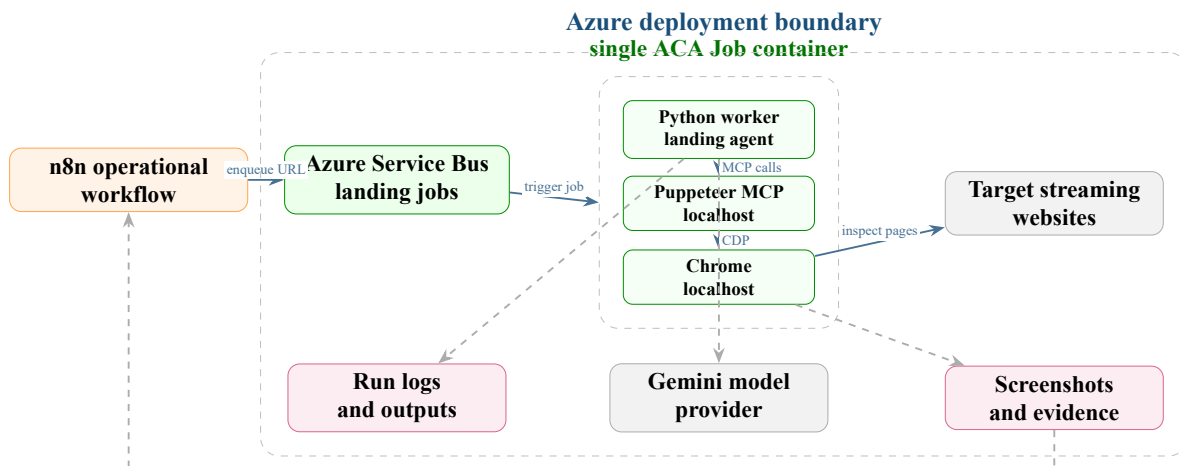


Figure 5.10: Azure Container Apps Job deployment using one container for worker, Puppeteer MCP, and Chrome.

Deployment element	Role	Scope during internship
n8n workflow	Keeps the company-preferred orchestration layer.	Main operational workflow remained in n8n.
Azure Service Bus	Decouples job submission from execution.	Used to pass landing-page jobs to the container worker.
Container Apps Job	Runs one containerized agent unit when a message arrives.	Single image for the Python worker, Puppeteer MCP server, and Chrome.
Puppeteer runtime	Opens the page and extracts match/host candidates through the local MCP endpoint.	Runs inside the same job container as the landing agent.
Outputs and artifacts	Return structured evidence, logs, and screenshots.	Used for review and integration back into the workflow.

Table 5.12: Landing-agent deployment scope and responsibilities.

The result is not only an extraction script. It is a reviewable agent platform where failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

6 Evaluation and Testing

This chapter evaluates the system as a browser-based evidence collection workflow. The goal is not only to classify a page correctly. A website is considered working only when the run reaches usable evidence: the page is classified, match or host candidates are extracted when the page is a landing page, the agent reaches a playable page, at least one networking or embedded media link is returned, and a screenshot shows the player state.

The evaluation is organized around three main points. First, agent evaluation measures whether the specialist agents can keep old websites working while new websites are added. Second, model evaluation compares model behavior, KPI stability, cache usage, and cost. Third, tool evaluation checks whether the browser tools extract the evidence they are supposed to extract. All three use the same regression idea: a change is accepted only if it improves new cases without breaking websites that previously worked.

6.1 Agent Evaluation and Website Regression

The main evaluation set is a batch dataset of more than 200 streaming websites. The dataset intentionally mixes languages, layouts, player types, redirect styles, and similar website families. This prevents the agents from being tuned only for one English layout or one player implementation. The same websites are rerun across prompt variations, model settings, and tool changes. Tests were run multiple times to reduce the risk of accepting a result caused by a lucky model response or temporary website state.

6.1.1 Test Protocol

The batch test follows the same sequence for each prompt, model, or tool change. First, the website list is loaded from the dataset. Each row contains a target URL and optional notes about language, layout, or expected page family. Second, the pipeline starts with classification and routes the URL to the proper specialist agent. Third, the specialist agents try to collect evidence: landing agents extract matches and host candidates, hosting agents activate players and switch servers, and embedded agents inspect iframes and harvest network media evidence. Fourth, the run output is checked against the acceptance rule. Finally, results are compared with previous runs for the same website.

This means the percentages in this chapter are not abstract model scores. They are website-batch scores. If a website is classified but no match, media link, or useful screenshot is returned, it does not count as a working website. Repetition is important because LLM decisions and streaming websites are both variable: a prompt or model change is trusted only when it survives repeated runs without losing previously working websites.

Test type	What is executed	What is recorded
Agent batch test	Run the complete pipeline on the 200+ website dataset.	Per-website pass/fail, page type, extracted matches, stream or embedded links, screenshot evidence, and failure reason.
Prompt regression test	Rerun the same dataset after changing agent instructions.	Websites newly fixed, websites still working, and websites that regressed.
Model comparison test	Rerun representative batches with the same prompt/tool setup but different model choices.	Website success rate, consistency across repeated runs, tokens, cache hit ratio, cost, and rate-limit behavior.
Tool regression test	Rerun the batch after changing browser tools or tool descriptions.	Whether inspect, navigate, interact, screenshot, harvest, and provider tools extract their intended evidence.
Manual evidence review	Inspect final traces, screenshots, stream URLs, and provider results for selected pass/fail cases.	Whether the automated pass/fail label is defensible and whether the failure belongs to prompt, model, tool, or website behavior.

Table 6.1: Evaluation test matrix executed over the website regression batch.

6.1.2 Batch Acceptance Rule

A website batch row is counted as successful only when the investigation returns enough evidence to be useful. The acceptance rule is stricter than a simple “run completed” status.

Checkpoint	Website is counted as working when
Page classification	The page type is correctly routed to landing, hosting, embedded, or other.
Match extraction	For landing pages, visible matches or event cards are extracted with usable links or host candidates.
Navigation	The agent reaches the next useful page instead of stopping at the first listing or ad page.
Player activation	The agent interacts with the player or server controls until a visible playback state, media state, or stream attempt is observed.
Network evidence	The run returns at least one useful network, embedded, manifest, or media URL candidate.
Screenshot evidence	A screenshot supports the final claim, preferably showing the player or the failure state that blocked playback.

Table 6.2: Website-level acceptance criteria for regression success.

The batch score is therefore calculated per website, not per tool call:

$$WebsiteSuccessRate = \frac{working_websites}{evaluated_websites}$$

$$RegressionRetention = \frac{previously_working_websites_still_working}{previously_working_websites}$$

The practical objective is to raise support for new websites while keeping *RegressionRetention* high. A prompt or tool change that supports a new player but breaks older working websites is treated as a regression.

6.1.3 Agent Use Cases

Each specialist agent is evaluated on the part of the investigation it owns. This makes failures easier to diagnose: a run can fail because the classifier routed incorrectly, the landing agent missed host candidates, the hosting agent failed to activate a server, or the embedded agent harvested too early.

Agent	Evaluated use cases	Success signal in the batch
Classification	Landing, hosting, embedded, unknown/other pages, redirects, blocked pages.	Correct route is selected and the next specialist receives the right task.
Landing page	Match cards, schedule lists, language-specific labels, tabs, pagination, and duplicate links.	Matches and host candidates are extracted without unrelated navigation noise.
Hosting page	Server buttons, play overlays, popups, ad tabs, redirects, and embedded handoff.	A server/player path is activated and a stream candidate or embedded player handoff is returned.
Embedded page	Iframes, player-only URLs, delayed manifests, tokenized stream URLs, and video state checks.	Network/media evidence is extracted and supported by a screenshot or clear failure reason.
Provider analysis	IPInfo, RDAP, WHOIS, Cloudflare-like providers, and abuse-contact metadata.	Extracted stream or host evidence can be enriched when provider data is available.

Table 6.3: Agent use cases and website-level success signals.

6.1.4 Prompt Regression Results

Prompt evaluation uses the same website batch for every prompt generation. The value in the final column is the percentage of websites in the batch that still met the acceptance rule after that prompt change. It is not an average number of tool calls.

The important trend is that v1 was not truly efficient; it simply stopped too early. The ReAct prompt improved behavior because it forced the agent to observe, act, and verify instead of guessing from the first page. Later versions improved the hard cases: popups, server switching, media-state checks, tokenized streams, and early exit after sufficient evidence was collected.

Version	Change tested	Regression focus	Batch websites working
v1	Intent-based prompt	Early stop, weak tab navigation, premature harvest.	38%
v2	ReAct loop and tab-check instruction	Navigation recall and screenshot-grounded decisions.	68%
v3	Popup handler and JavaScript click fallback	Popup handling and server activation after ad traps.	74%
v4	Mandatory media-state check before harvest	Player activation and premature harvest prevention.	81%
v5	Tool budget cap and early success exit	Cost reduction without losing previously working websites.	83%

Table 6.4: Prompt regression history and batch-level behavioral focus.

Metric	Intent v1	ReAct v2	Current v3/v5
Page-type routing accuracy	91%	93%	95%
Host-candidate recall	40%	85%	89%
Player activation success	0%	70%	90%
Popup-interference recovery	20%	55%	78%
Tokenized-stream recovery	25%	40%	85%
Manifest recovery on AlbaPlayer case	0%	70%	90%
Manifest recovery on tokenized Clappr case	—	40%	85%

Table 6.5: Use-case KPI results measured on the website regression batch.

6.1.5 Dashboard Snapshot

The dashboard is used to inspect what the batch execution produced. It is not only a visual summary; it is the place where run, token, cost, provider, tool, and agent telemetry are checked after experiments. The screenshots used for this report show a larger dashboard snapshot than the earlier local test database: model costs, tokens, provider lookups, and tool health are all visible.

These values complement the website-batch scores. The batch score says whether a website worked. The dashboard explains why a run was expensive, whether caching helped, which models were used, which tools failed, and whether extracted stream URLs could be enriched with provider information.

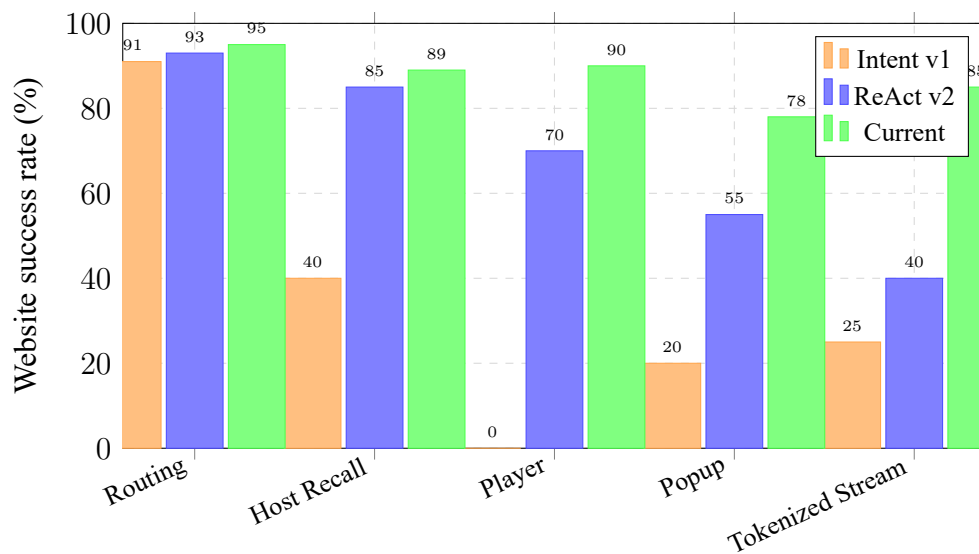


Figure 6.1: Regression-batch improvement across the main browser-agent KPIs.

Dashboard area	What it validates	Observed value
Cost telemetry	Model-spend aggregation and average cost per run.	Total cost: \$4.505017; average cost/run: \$0.047926.
LLM calls	Whether the batch persisted model invocations.	883 calls; 8 calls/run.
Token telemetry	Whether input, cached input, and output tokens are separated.	Total: 40,220,868; output: 454,914.
Cache behavior	Whether repeated prompt/tool context is being reused.	Cached input: 28,928,333; cache hit rate: 40.0%.
Provider intelligence	Whether extracted stream URLs can be resolved to infrastructure metadata.	694 checked URLs, 661 resolved IPs, 637 provider matches.
Tool health	Whether browser tool execution is stable during runs.	1,410 calls, 96.7% success, 46 failed calls.

Table 6.6: Dashboard telemetry values used as operational evidence.

6.2 Model Evaluation, KPI Selection, and Cache

Model evaluation uses the same 200+ website batch and the same prompt/tool configuration for each model candidate. This keeps the comparison fair: the metric should reflect model behavior, not a different browser policy or a different prompt. Each model is evaluated on quality, consistency, cost, cache behavior, and rate-limit practicality.

6.2.1 KPI Formulas

The evaluation uses website-level KPI formulas. Let N be the number of evaluated websites in the batch, W the number of websites that satisfy the acceptance rule, C the number correctly classified, H the number where valid host candidates are found, P the number where the player is activated, and E the number where media/network evidence is returned.

KPI	Formula and meaning
Website success rate	W/N . Main regression metric: the percentage of sites that still work end-to-end.
Classification accuracy	C/N . Measures whether the correct specialist route is selected.
Host-candidate recall	valid host candidates found divided by manually expected host candidates for landing pages.
Player activation rate	P / activation attempts. A success requires playback state, media state, or screenshot-supported stream attempt.
Evidence extraction rate	E/N . Measures websites returning at least one network, manifest, embedded, or media URL candidate.
Regression retention	previously working websites still working divided by previously working websites.
Consistency rate	websites with the same pass/fail result across repeated runs divided by repeated websites.
Cache hit ratio	$cached_input_tokens / (cached_input_tokens + new_input_tokens)$.
Cost per working website	total estimated model cost divided by websites that satisfy the acceptance rule.

Table 6.7: KPI formulas for model, agent, and website-batch evaluation.

6.2.2 Model Selection Criteria

A stronger model is not selected only because it produces longer reasoning. It must improve the website-level score while staying operationally usable.

6.2.3 LLM Model Comparison

Several model families were tested before selecting the final runtime model. The final choice was `google_genai::gemini-3.1-flash-lite`. The decision was not based on one benchmark number. It came from batch behavior: the model had to keep enough context for long

Dimension	What is measured	Why it affects selection
Accuracy	Website success rate, classification accuracy, evidence extraction rate.	The selected model must increase successful sites, not just produce confident text.
Consistency	Same website and prompt run multiple times.	Streaming sites are noisy; the model should not randomly lose working websites.
Tool discipline	Correct use of inspect, interact, navigate, screenshot, and harvest tools.	Bad tool order can break playback even when reasoning looks correct.
Context handling	Ability to preserve page state, server attempts, iframe context, and failure reasons.	Long browser traces require stable state tracking.
Cache efficiency	Cache hit ratio and cached-token cost reduction.	Repeated prompts and tool descriptions should be reused instead of billed as new input every run.
Rate limits	Requests per minute, tokens per minute, and requests per day.	A model that works once may still be unusable for large batch evaluation.

Table 6.8: Model selection criteria for the regression batch.

browser traces, use tools reliably, respond quickly, stay cheap during repeated website tests, and support caching well enough for large prompt/tool context.

Gemini 2.5 Flash-Lite was inexpensive but weak for this workflow: it often stopped early, missed page state, or failed to reason through chained navigation. Gemini 2.5 Flash was stronger and reasoned better, but it was more expensive for repeated batch evaluation. OpenRouter free models were useful for exploration, but most were not reliable enough for the main pipeline. GLM 4.5 Air, Devstral 2512, NVIDIA Nemotron 3 Super, GPT-OSS 120B, DeepSeek V3.2, and MiniMax M2.5 were tested, but the free endpoints were often slower, less consistent, or constrained by smaller effective context windows and routing limits. For this project, a model that is occasionally strong is less useful than a model that is fast, cheap, repeatable, and can handle long context every run.

Table 6.9: LLM model comparison.

Model tested	Ctx.	In / 1M	Cache in / 1M	Out / 1M	Evaluation note
Gemini 3.1 Flash-Lite	1,048k	\$0.25	\$0.025	\$1.50	Final choice: 1M context, fast inference, cheap operation, good cache efficiency, and generous operating limits.

Model tested	Ctx.	In / 1M	Cache in / 1M	Out / 1M	Evaluation note
Gemini 2.5 Flash-Lite	1,048k	\$0.10	\$0.010	\$0.40	Cheap, but performed poorly on multi-step navigation and tool-grounded reasoning.
Gemini 2.5 Flash	1,048k	\$0.30	\$0.030	\$2.50	Good reasoning and stronger agent behavior, but higher cost for large repeated batches.
GLM 4.5 Air free	131k	\$0.00	–	\$0.00	Useful for trials, but constrained by smaller context and free-tier routing.
GLM 4.5 Air paid	131k	\$0.13	\$0.025	\$0.85	Reasonable price, but not selected because context was smaller than Gemini.
Devstral 2512	262k	\$0.40	\$0.040	\$2.00	Tested for coding-style reasoning, but not stable enough for the browser pipeline.
NVIDIA Nemotron 3 Super free	1,000k	\$0.00	–	\$0.00	Attractive context, but free route behavior was less predictable for repeated runs.
NVIDIA Nemotron 3 Super paid	1,000k	\$0.09	–	\$0.45	Cheap long-context option, but final pipeline used Gemini because of speed and integration behavior.
GPT-OSS 120B free	131k	\$0.00	–	\$0.00	Free, but small context for this workload and less reliable in browser-agent traces.
GPT-OSS 120B paid	131k	\$0.039	–	\$0.18	Very cheap, but context and behavior were not the best fit.
DeepSeek V3.2	131k	\$0.252	\$0.0252	\$0.378	Tested, but not selected because long browser traces need larger context.
MiniMax M2.5 free	205k	\$0.00	–	\$0.00	Painfully slow inference and weak fit for fast browser loops.
MiniMax M2.5 paid	205k	\$0.15	–	\$1.15	Paid endpoint did not justify switching from Gemini for this workflow.

Prices and context windows in Table 6.9 are from OpenRouter’s Models API, where pricing is reported per token and converted here to USD per million tokens [8]. The cache column uses OpenRouter’s `input_cache_read` field when available; -- means that no cache-read price was exposed for that model.

Google’s Gemini pricing and caching documentation separates normal input, cached input, and output pricing, while the rate-limit documentation describes RPM, TPM, and RPD as operational limits that vary by model and usage tier [10, 11, 9]. Those properties match the project constraints: the agents need a large context window for page state and traces, low latency for browser loops, low cost for repeated 200+ website regression batches, and enough rate-limit

headroom for repeated experiments. That is why `google_genai:gemini-3.1-flash-lite` became the default model.

6.2.4 Cost Estimation

Cost is estimated from token usage and model pricing. Cached tokens are separated from new input tokens because provider-side caching changes the real price of repeated prompt and tool context.

$$\begin{aligned} \text{Cost} = & \frac{\text{new_input_tokens}}{1,000,000} \times \text{input_price} \\ & + \frac{\text{cached_input_tokens}}{1,000,000} \times \text{cached_input_price} \\ & + \frac{\text{cache_write_tokens}}{1,000,000} \times \text{cache_write_price} \\ & + \frac{\text{output_tokens}}{1,000,000} \times \text{output_price} \end{aligned}$$

$$\text{CostPerWorkingWebsite} = \frac{\text{total_estimated_cost}}{\text{working_websites}}$$

The dashboard exposes input cost, cached-input cost, cache-write cost, output cost, and total cost. In the dashboard snapshot, the model-spend view reports \$4.505017 total recorded spend, \$0.047926 average cost per run, 883 LLM calls, four provider names, and a blended cost of \$0.000112 per 1k tokens.

Cost / token metric	Interpretation	Dashboard value
Total model spend	Sum of recorded or computed spend across model usage rows.	\$4.505017
Average cost per run	Total spend divided by persisted runs with usage.	\$0.047926
LLM calls	Number of model invocations recorded during the experiment period.	883
Average LLM calls per run	Approximate model invocations needed by a run.	8
Blended cost per 1k tokens	Total spend normalized by token volume.	\$0.000112
Total displayed tokens	Dashboard token total for input and output usage.	40,220,868
New input tokens	Non-cached prompt/context tokens.	43,384,638
Cached input tokens	Input tokens served through prompt cache.	28,928,333
Output tokens	Generated model tokens.	454,914
Cache hit rate	Cached input divided by total input considered by the cache view.	40.0%
Average tokens per run	Token volume normalized by persisted run count.	386,739

Table 6.10: Observed cost, token, and cache metrics from the dashboard.

The important point is that cache is evaluated as part of model selection. A model that is accurate but repeatedly consumes full prompt and tool context can become too expensive for large batch testing. A useful configuration keeps the website success rate stable while raising the cache hit rate and lowering cost per working website.

6.2.5 Provider Intelligence Test

Provider analysis is evaluated after stream or embedded URLs are collected. The test takes the extracted URLs, resolves their hostnames, checks whether a routable IP is found, and then enriches the result with provider and country information using IP and registration data sources such as IPinfo and RDAP/WHOIS lookups [31, 32]. This matters because a stream URL is stronger evidence when it can be connected to a hosting provider, CDN, or network operator.

Provider metric	Meaning	Dashboard value
Checked URLs	Stream or media URLs inspected by the provider lookup flow.	694
Resolved IPs	URLs that resolved to a routable IP address.	661
Provider matches	URLs mapped to an organization or hosting provider.	637
Abuse contacts	Abuse email contacts found through the lookup path.	0
Provider coverage	Provider matches divided by checked URLs.	$637 / 694 = 91.8\%$
IP resolution coverage	Resolved IPs divided by checked URLs.	$661 / 694 = 95.2\%$

Table 6.11: Provider-intelligence telemetry used to validate stream URL enrichment.

The dashboard also shows the distribution of infrastructure behind the extracted URLs. The largest providers in the observed snapshot were DDOS-GUARD LTD, StormWall s.r.o., Fastly, Cloudflare, SpectraIP B.V., Akamai, Private Layer INC, and Amazon. Country distribution was also recorded, with the largest groups in Russia, the United States, and the Netherlands. These values do not decide whether a website works, but they validate the post-extraction enrichment layer.

6.3 Tool Evaluation

Tool evaluation also uses the website batch. The question is whether a tool extracts what it was designed to extract on real website layouts. A tool is useful only if it helps the agent satisfy the website acceptance rule: classify, navigate, activate playback, return network evidence, and capture a screenshot-supported result.

6.3.1 Tool Success Criteria

The tool dashboard is used to verify that this layer is not silently failing while the agent is reasoning. In the observed dashboard, the browser/tool layer recorded 1,410 tool calls with a

Tool family	Expected extraction	Batch success signal
Inspect tools	Compact DOM structure, visible buttons, links, match cards, iframes, player controls, and page hints.	Agent receives enough context to choose the next correct action without overloading the model.
Navigation tools	Open target URLs, follow host candidates, keep same-page tab state when needed.	Agent reaches the correct landing, hosting, or embedded page.
Interaction tools	Click play, close popups, switch servers, activate tabs, restore focus after ad windows.	Player state changes or a meaningful blocker is documented.
Screenshot tools	Capture visual state at classification, player activation, and final evidence moments.	Screenshot supports why the website passed or failed.
Harvest tools	Collect network entries, media URLs, iframe URLs, manifests, and performance evidence.	At least one useful network, embedded, or media candidate is returned when playback is available.
Provider tools	Enrich host and stream evidence with IPInfo, RDAP, WHOIS, or provider metadata.	Evidence can be linked to provider or abuse-contact context when available.

Table 6.12: Tool-family evaluation criteria for batch-level testing.

96.7% success rate. There were 46 failed calls, eight distinct tools, an average duration of 6.51 seconds per call, and 14 tool calls per pipeline run.

Tool metric	What it means	Dashboard value
Total calls	Total observed browser/tool invocations.	1,410
Successful calls	Tool calls that returned a successful outcome.	1,364
Failed calls	Tool calls that returned an error or failed outcome.	46
Tool success rate	Successful calls divided by total calls.	96.7%
Unique tools	Distinct tool names observed in the dashboard.	8
Average duration	Mean observed duration per tool call.	6.51 s
Calls per run	Average tool calls per pipeline run.	14

Table 6.13: Observed tool-health telemetry from the dashboard.

Tool success rate alone is not enough, because a tool can technically succeed while extracting the wrong evidence. For example, a successful screenshot call does not mean the player was activated. Therefore tool evaluation combines dashboard health with website acceptance: the tool must return the expected kind of evidence and the website must still pass the batch rule.

6.3.2 Tool Regression Findings

The main tool failures were behavioral. The tools often worked technically, but the agent used them at the wrong time or for the wrong page type. Regression testing catches these cases because a website that previously returned evidence stops passing the batch.

Observed issue	Regression caused	Fix applied
harvest() before play click	Website fails because no runtime stream has been generated yet.	Added activation guard and mandatory media-state check.
Generic inspect on hosting pages	Server controls, tabs, and player-specific elements are missed.	Added page-profiled inspect behavior and clearer tool descriptions.
Repeated screenshots as waiting	Higher cost without new evidence and no playback progress.	Added explicit wait/checkpoint logic and reduced screenshot loops.
navigate() for same-page tabs	Page reloads or loses state instead of opening dynamic server content.	Clarified interact versus navigate.
Ignoring popup or new-tab focus	Agent reasons over the advertisement page instead of the original player page.	Added popup close, focus restore, and JavaScript-click fallback.

Table 6.14: Tool-level regressions found through website-batch testing.

Tool change	Before	After	Meaning
Activation guard on harvest	65% misuse	12% misuse	Fewer websites failed because harvesting happened before playback.
Specialized hosting inspection	40% missed controls	8% missed controls	Server buttons and player controls were found more reliably.
Clarified navigate versus interact	30% misuse	5% misuse	Same-page tabs and dynamic controls were handled without unnecessary reloads.

Table 6.15: Tool-description improvements measured against the website batch.

6.4 Chapter Conclusion

The evaluation shows that the system must be judged by website-level evidence collection, not by isolated tool counts or a single classifier score. A website works only when the agents classify it, navigate through its layout, activate or inspect the player, recover from popups or redirects, extract media/network evidence, and return screenshot-supported results.

The 200+ website batch is the central evaluation mechanism. It supports regression testing because every prompt, model, and tool change is checked against old and new websites. This matches the real engineering goal: add support for more layouts without losing sites that already worked.

Model evaluation adds another layer by comparing accuracy, consistency, cache hit ratio, rate limits, and cost per working website. Tool evaluation checks whether each browser tool extracts the evidence it is responsible for on real pages. Together, these three evaluation points provide a practical way to improve the system without relying on one-off successful demos.

7 Conclusion and Perspectives

7.1 General Conclusion

This internship produced an agentic evidence-collection system for illegal sports streaming websites. The work started from a practical problem: streaming-piracy pages do not expose evidence through simple static HTML. They use dynamic listings, JavaScript tabs, popup traps, nested iframes, click-gated players, redirects, provider checks, and short-lived stream URLs. The final system therefore had to behave more like a controlled browser investigator than a normal scraper.

The project followed a general-to-specific progression. The n8n phase validated the core idea quickly: an LLM-guided browser workflow can inspect pages, follow page transitions, activate players, and recover evidence. The Python reference implementation then turned that prototype into a more maintainable architecture with FastAPI, a Next.js operator console, profiled MCP browser tools, specialist agents, memory and cache support, PostgreSQL/SQLite observability, and cost-aware model telemetry.

The most important result is not a single extraction trick. It is a repeatable engineering framework for investigating unstable websites while preserving reviewable evidence: traces, screenshots, stream candidates, page classifications, model/tool usage, and failure reasons.

7.2 Main Contributions

Contribution	Summary
n8n feasibility prototype	Validated the agentic browser workflow on real dynamic pages and exposed the limitations of long sequential visual workflows.
Specialist agent design	Split the task into classification, landing, hosting, and embedded agents, each with a focused prompt, tool profile, and stopping behavior.
Browser-tool layer	Built Puppeteer/MCP tools for inspection, interaction, navigation, screenshot capture, media-state checks, stream harvesting, and memory support.
Operator platform	Provided a FastAPI backend and Next.js console for run launching, run review, event streaming, screenshots, and dashboard metrics.
Evaluation framework	Defined KPI formulas for classification, navigation, player activation, stream extraction, popup handling, tool behavior, cache, and cost.
Observability and cost tracking	Persisted run records, events, model usage, tool calls, cache fields, and cost fields so failures can be diagnosed instead of guessed.

Table 7.1: Final summary of project contributions and outcomes.

7.3 Technical Lessons

7.3.1 Prompt structure matters

The largest reliability gain came from replacing broad intent-based prompts with ReAct-style prompts. The agent needed to observe, state a hypothesis, choose one action, verify the result, and then continue or stop. Without that loop, the system stopped too early, missed hidden tabs, harvested before activation, or accepted weak evidence.

7.3.2 Browser evidence must be layered

DOM text alone was not enough. A valid result needed several evidence layers: visible state, screenshot context, tool trace, media state, stream URL, iframe path, and final structured output. This is why the implementation treats screenshots and runtime events as core evidence rather than debugging extras.

7.3.3 Fallbacks are part of the architecture

Streaming websites rarely follow one clean path. Landing pages may link to host pages, host pages may redirect to embedded players, and embedded players may be nested several frames deep. The hosting-to-embedded fallback was therefore essential. It allowed the system to recover cases that a single-stage scraper would mark as failed.

7.3.4 Evaluation must be regression-oriented

The evaluation showed that every improvement can create a new failure. More tool calls can improve recovery, but also increase cost. More aggressive ad blocking can reduce noise, but can also break a player. A stronger prompt can improve one page type while overfitting another. The correct evaluation method is therefore regression testing across stable use cases: classification, landing navigation, player activation, server switching, popup handling, redirect handling, stream extraction, and metadata enrichment.

7.4 Limitations

The system remains a reference architecture, not a finished production enforcement platform. The main limitations are:

- anti-bot and provider behavior changes continuously;
- some runs still require human review before the evidence can be trusted;
- the local dashboard snapshot is small and does not yet represent production-scale accuracy;
- model and pricing telemetry must be collected over more real runs before a true LLM model leaderboard can be claimed;
- deeply nested iframe chains, token expiry, and popup-heavy pages still require careful budget and retry policies.

These limitations do not invalidate the approach. They define the next engineering steps: larger evaluation sets, more stable telemetry capture, clearer model comparisons, and stronger run-level validation.

7.5 Future Work

The most useful short-term improvements are practical:

- expand the fixed benchmark set with labeled landing, hosting, embedded, popup, redirect, and tokenized-stream cases;
- persist model/tool telemetry consistently for every dashboard run so cost, cache, and model comparisons are always available;
- add automated regression suites for each agent and each page behavior;
- improve candidate ranking so the system spends less time on low-quality host pages and dead server buttons;
- refine memory so successful selectors, server labels, and provider patterns can be reused without hardcoding websites;
- move long browser work to queue-driven workers while keeping the operator console responsive.

7.5.1 Screenshot annotation layer

A useful operator-console extension is a canvas-based screenshot annotation layer. The system already captures screenshots; the next step is to let reviewers mark evidence regions directly: server buttons, play controls, iframe containers, blocking overlays, broadcaster logos, player state, and stream-relevant UI areas.

Each annotation can be stored as structured metadata linked to the run, screenshot URL, page URL, timestamp, and reviewer. This would improve human review, make report figures clearer, and support evidence handoff without modifying the target website.

7.6 Final Remarks

The project demonstrates that agentic browser automation is a suitable approach for this domain when it is constrained, observable, and evaluated carefully. The final value is not that the system permanently defeats every anti-bot technique. The value is that it turns a messy investigation process into a structured workflow: classify the page, navigate deliberately, activate and verify the player, extract evidence, preserve the trace, estimate the cost, and learn from failures.

That framework gives Soft Stars a foundation for continuing the work: more data, more evaluation, more robust deployment, and clearer operational review.

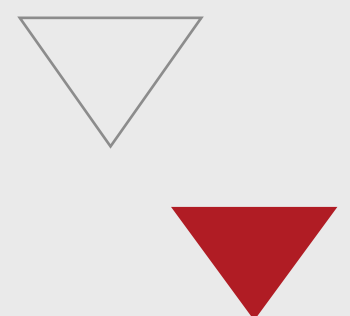
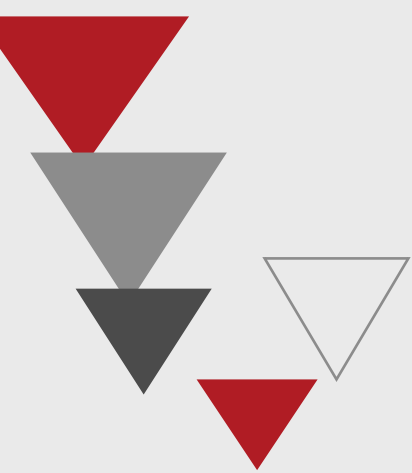
References

Bibliography

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. Available: <https://arxiv.org/abs/2210.03629>
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Ciaramita, L. Zettlemoyer, H. Schütze, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. Available: <https://arxiv.org/abs/2302.04761>
- [3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A Survey on Large Language Model based Autonomous Agents,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024. Available: <https://arxiv.org/abs/2308.11432>
- [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: <https://arxiv.org/abs/2201.11903>
- [5] Anthropic, *Claude 3 Model Card*, Anthropic, 2024. Available: <https://www.anthropic.com/claude>
- [6] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, Technical Report, Google DeepMind, 2023. Available: <https://arxiv.org/abs/2312.11805>
- [7] Google, *Gemini 3.1 Flash-Lite: Built for Intelligence at Scale*, The Keyword, 2026. Available: <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-flash-lite/>
- [8] OpenRouter, *Models API*, Official documentation, 2026. Available: <https://openrouter.ai/api/v1/models>
- [9] Google, *Gemini API Rate Limits*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/rate-limits>
- [10] Google, *Gemini Developer API Pricing*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [11] Google, *Context Caching in the Gemini API*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/caching>
- [12] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: <https://pptr.dev>
- [13] Microsoft, *Playwright Documentation*, Official documentation, 2026. Available: <https://playwright.dev/docs/intro>

- [14] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A Realistic Web Environment for Building Autonomous Agents,” in *Proceedings of ICLR*, 2024. Available: <https://arxiv.org/abs/2307.13854>
- [15] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a Generalist Agent for the Web,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [17] T. Le Sellier de Chezelles, M. Gasse, A. Drouin, and others, “The BrowserGym Ecosystem for Web Agent Research,” *arXiv preprint arXiv:2412.05467*, 2024.
- [18] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [19] R. Wirth and J. Hipp, “CRISP-DM: Towards a Standard Process Model for Data Mining,” in *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining (PAKDD)*, 2000, pp. 29–39.
- [20] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: <https://www.jstor.org/stable/25148625>
- [21] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: <https://doi.org/10.2753/MIS0742-1222240302>
- [22] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: <https://fastapi.tiangolo.com>
- [23] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: <https://nextjs.org/docs>
- [24] LangChain, *LangChain Agents and Tools Documentation*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langchain/agents>
- [25] LangChain, *LangGraph Overview*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [26] Model Context Protocol, *Model Context Protocol Specification*, Official documentation, 2026. Available: <https://modelcontextprotocol.io/specification/2024-11-05/basic/index>
- [27] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: <https://docs.n8n.io>
- [28] Microsoft, “Jobs in Azure Container Apps,” *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/container-apps/jobs>

- [29] Microsoft, “Azure Service Bus Messaging Service,” *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>
- [30] Cloudinary, *Cloudinary Upload API Documentation*, Official documentation, 2026. Available: <https://cloudinary.com/documentation>
- [31] IPinfo, *IPinfo API Documentation*, Official documentation, 2026. Available: <https://ipinfo.io/developers/ipinfo-api>
- [32] ICANN, *Registration Data Access Protocol (RDAP)*, Official documentation, 2026. Available: <https://www.icann.org/rdap/>
- [33] Cloudflare, *How Cloudflare Challenges Work*, Official documentation, 2026. Available: <https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/>
- [34] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: <https://www.rfc-editor.org/rfc/rfc8216>
- [35] ISO/IEC, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO/IEC 23009-1, 2022.
- [36] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: <https://github.com/gorhill/uBlock>
- [37] MUSO, *Global Piracy Insights: Sports Streaming Report*, MUSO Intelligence, 2023. Available: <https://www.muso.com/insights>
- [38] European Audiovisual Observatory, *Trends in the Field of Audiovisual Piracy in Europe*, Council of Europe, Strasbourg, 2022. Available: <https://rm.coe.int/>



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454